

L-BFGS-B

3.0

Generated by Doxygen 1.9.1

1 L-BFGS-B	1
2 File Index	3
2.1 File List	3
3 File Documentation	4
3.1 src/active.f File Reference	4
3.1.1 Function/Subroutine Documentation	4
3.2 src/bmv.f File Reference	5
3.2.1 Function/Subroutine Documentation	5
3.3 src/cauchy.f File Reference	6
3.3.1 Function/Subroutine Documentation	6
3.4 src/cmprlb.f File Reference	10
3.4.1 Function/Subroutine Documentation	10
3.5 src/dcsrch.f File Reference	11
3.5.1 Function/Subroutine Documentation	12
3.6 src/dcstep.f File Reference	14
3.6.1 Function/Subroutine Documentation	14
3.7 src/errclb.f File Reference	15
3.7.1 Function/Subroutine Documentation	16
3.8 src/formk.f File Reference	16
3.8.1 Function/Subroutine Documentation	17
3.9 src/format.f File Reference	18
3.9.1 Function/Subroutine Documentation	19
3.10 src/freew.f File Reference	19
3.10.1 Function/Subroutine Documentation	20
3.11 src/hpsolb.f File Reference	21
3.11.1 Function/Subroutine Documentation	21
3.12 src/lnsrbl.f File Reference	22
3.12.1 Function/Subroutine Documentation	22
3.13 src/mainlb.f File Reference	24
3.13.1 Function/Subroutine Documentation	24
3.14 src/matupd.f File Reference	29
3.14.1 Function/Subroutine Documentation	29
3.15 src/prn1lb.f File Reference	30
3.15.1 Function/Subroutine Documentation	31
3.16 src/prn2lb.f File Reference	32
3.16.1 Function/Subroutine Documentation	32
3.17 src/prn3lb.f File Reference	33
3.17.1 Function/Subroutine Documentation	34
3.18 src/projgr.f File Reference	35
3.18.1 Function/Subroutine Documentation	35
3.19 src/setulb.f File Reference	36

3.19.1 Function/Subroutine Documentation	36
3.20 src/subsm.f File Reference	40
3.20.1 Function/Subroutine Documentation	40
3.21 src/timer.f File Reference	43
3.21.1 Function/Subroutine Documentation	43
4 Example Documentation	44
4.1 driver1.f	44
4.2 driver1.f90	48
4.3 driver2.f	52
4.4 driver2.f90	55
4.5 driver3.f	58
4.6 driver3.f90	61
Index	65

1 L-BFGS-B

Software for Large-scale Bound-constrained Optimization

L-BFGS-B is a limited-memory quasi-Newton code for bound-constrained optimization, i.e., for problems where the only constraints are of the form $l \leq x \leq u$. It is intended for problems in which information on the Hessian matrix is difficult to obtain, or for large dense problems. **L-BFGS-B** can also be used for unconstrained problems, and in this case performs similarly to its predecessor, algorithm **L-BFGS** (Harwell routine VA15). The algorithm is implemented in Fortran 77.

Authors

- Ciyou Zhu
- Richard Byrd
- Jorge Nocedal
- Jose Luis Morales

Related Publications

- R. H. Byrd, P. Lu, J. Nocedal and C. Zhu. **A Limited Memory Algorithm for Bound Constrained Optimization** (1995), SIAM Journal on Scientific and Statistical Computing, Vol. 16, Num. 5, pp. 1190-1208
- C. Zhu, R. H. Byrd and J. Nocedal. **L-BFGS-B: Algorithm 778: L-BFGS-B, FORTRAN routines for large scale bound constrained optimization** (1997), ACM Transactions on Mathematical Software, Vol. 23, Num. 4, pp. 550-560
- J.L. Morales and J. Nocedal. **L-BFGS-B: Remark on Algorithm 778: L-BFGS-B, FORTRAN routines for large scale bound constrained optimization** (2011), ACM Transactions on Mathematical Software, Vol. 38, Num. 1

- R. H. Byrd, J. Nocedal and R. B. Schnabel. [Representations of quasi-Newton matrices and their use in limited memory methods](#) (1994), Mathematical Programming, Vol. 63, pp. 129-156

Note that the subspace minimization in the [LBFGSpp](#) implementation is an exact minimization subject to the bounds, based on the BOXCQP algorithm:

- C. Voglis and I. E. Lagaris, [BOXCQP: An Algorithm for Bound Constrained Convex Quadratic Problems](#) (2004), 1st International Conference "From Scientific Computing to Computational Engineering", Athens, Greece

For an eagle-eye overview of L-BFGS-B and the genealogy BFGS->L-BFGS->L-BFGS-B, see [Henao's Master's thesis](#).

Related Software

- [wilmerhenao/L-BFGS-B-NS](#): An L-BFGS-B-NS Optimizer for Non-Smooth Functions
- [pcarbo/lbfgsb-matlab](#): A MATLAB interface for L-BFGS-B
- [bgranzow/L-BFGS-B](#): A pure Matlab implementation of L-BFGS-B (LBFGSB)
- [constantino-garcia/lbfgsb_cpp_wrapper](#): A simple C++ wrapper around the original Fortran L-BFGS-B routine
- [yixuan/LBFGSpp](#): A header-only C++ library for L-BFGS and L-BFGS-B algorithms
- [chokkan/liblbfgs](#): libLBFGS: a library of Limited-memory Broyden-Fletcher-Goldfarb-Shanno (L-BFGS)
- [mkobos/lbfgsb_wrapper](#): Java wrapper for the Fortran L-BFGS-B algorithm
- [yuhonglin/Lbfgsb.jl](#): A Julia wrapper of the l-bfgs-b fortran library
- [Gnimuc/LBFGSB.jl](#): Julia wrapper for L-BFGS-B Nonlinear Optimization Code
- [afbarnard/go-lbfgsb](#): L-BFGS-B optimization for Go, C, Fortran 2003
- [nepluno/lbfgsb-gpu](#): An open source library for the GPU-implementation of L-BFGS-B algorithm
- [Chris00/L-BFGS-ocaml](#): OCaml bindings for L-BFGS
- [dwicke/L-BFGS-B-Lua](#): L-BFGS-B lua wrapper around a L-BFGS-B C implementation
- [avieira/python_lbfgsb](#): Pure Python-based L-BFGS-B implementation
- [ybyygu/rust-lbfgsb](#): Ergonomic bindings to L-BFGS-B code for Rust
- [rforge/lbfgsb3c](#): Limited Memory BFGS Minimizer with Bounds on Parameters with optim() 'C' Interface for R
- [florafauna/optimParallel-python](#): A parallel version of 'scipy.optimize.minimize(method='L-BFGS-B')

Notes on this repository

I (J. Schilling) took the freedom to

- put the L-BFGS-B code obtained from [the original website](#) up in this repository,
- divide the subroutines and functions into separate files,
- convert parts of the documentation into a format understandable to [doxygen](#) and
- replace the included BLAS/LINPACK routines with calls to user-provided BLAS/LAPACK routines. To be precise, the calls to LINPACK's `dtrsl` were replaced with calls to LAPACK's `dtrsm` and the calls to LINPACK's `dpofa` were replaced with calls to LAPACK's `dpotrf`.

Building

A CMake setup is provided for L-BFGS-B in this repository. External modules for BLAS and LAPACK have to be installed on your system. Then, building the shared library `liblbfgsb.so` and the examples `driver*.f` and `driver*.f90` works as follows:

```
> mkdir build
> cd build
> cmake ..
> make -j
```

The resulting shared library and the driver executables can be found in the `build` directory.

Concluding remarks

The current release is version 3.0. The distribution file was last changed on 02/08/11.

This work was in no way intending to infringe any copyrights or take credit for others' work. Feel free to contact me at any time in case you noticed something against the rules. Above documentation is obtained from the archived version of the [original manual](#).

A PDF version of the documentation is available here: [L-BFGS-B.pdf](#).

2 File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

src/active.f	4
src/bmv.f	5
src/cauchy.f	6
src/cmprlb.f	10
src/dcsrch.f	11
src/dcstep.f	14

src/errclb.f	15
src/formk.f	16
src/format.f	18
src/freev.f	19
src/hpsolb.f	21
src/lnsr1b.f	22
src/main1b.f	24
src/matupd.f	29
src/prn11b.f	30
src/prn21b.f	32
src/prn31b.f	33
src/projgr.f	35
src/setulb.f	36
src/subsm.f	40
src/timer.f	43

3 File Documentation

3.1 src/active.f File Reference

Functions/Subroutines

- subroutine [active](#) (n, l, u, nbd, x, iwhere, iprint, prjctd, cnstnd, boxed)
This subroutine initializes iwhere and projects the initial x to the feasible set if necessary.

3.1.1 Function/Subroutine Documentation

3.1.1.1 active() subroutine active (
integer n,
double precision, dimension(n) l,
double precision, dimension(n) u,
integer, dimension(n) nbd,
double precision, dimension(n) x,
integer, dimension(n) iwhere,
integer iprint,
logical prjctd,
logical cnstnd,
logical boxed)

This subroutine initializes iwhere and projects the initial x to the feasible set if necessary.

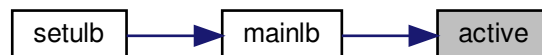
Parameters

<i>n</i>	number of parameters
<i>l</i>	lower bounds on parameters
<i>u</i>	upper bounds on parameters
<i>nbd</i>	indicates which bounds are present
<i>x</i>	position
<i>iwhere</i>	On entry <i>iwhere</i> is unspecified. On exit: <i>iwhere</i> (<i>i</i>)= • -1 if <i>x</i>(<i>i</i>) has no bounds • 3 if <i>l</i>(<i>i</i>)=<i>u</i>(<i>i</i>), • 0 otherwise. In <i>cauchy</i> , <i>iwhere</i> is given finer gradations.
<i>iprint</i>	console output flag
<i>prjctd</i>	On exit <i>.true.</i> if any input <i>x</i> (<i>i</i>) had to be clipped onto its bound (the user supplied an infeasible starting point).
<i>cnstnd</i>	On exit <i>.true.</i> if any variable has at least one bound (any <i>nbd</i> (<i>i</i>) $\neq$ 0); <i>.false.</i> for purely unconstrained problems.
<i>boxed</i>	On exit <i>.true.</i> iff every variable has both lower and upper bounds (every <i>nbd</i> (<i>i</i>) == 2); <i>.false.</i> otherwise.

Definition at line 35 of file *active.f*.

Referenced by *mainlb*().

Here is the caller graph for this function:



3.2 src/bmv.f File Reference

Functions/Subroutines

- subroutine *bmv* (*m*, *sy*, *wt*, *col*, *v*, *p*, *info*)

This subroutine computes the product of the 2m x 2m middle matrix in the compact L-BFGS formula of B and a 2m vector v.

3.2.1 Function/Subroutine Documentation

```

3.2.1.1 bmv() subroutine bmv (
    integer m,
    double precision, dimension(m, m) sy,
    double precision, dimension(m, m) wt,
    integer col,
    double precision, dimension(2*col) v,
    double precision, dimension(2*col) p,
    integer info )

```

This subroutine computes the product of the 2m x 2m middle matrix in the compact L-BFGS formula of B and a 2m vector v; it returns the product in p.

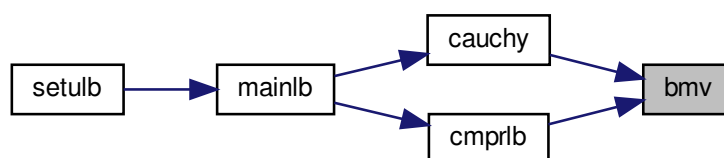
Parameters

<i>m</i>	On entry m is the maximum number of variable metric corrections used to define the limited memory matrix. On exit m is unchanged.
<i>sy</i>	On entry sy specifies the matrix S'Y. On exit sy is unchanged.
<i>wt</i>	On entry wt specifies the upper triangular matrix J' which is the Cholesky factor of $(\theta S'S + LD^{(-1)}L')$. On exit wt is unchanged.
<i>col</i>	On entry col specifies the number of s-vectors (or y-vectors) stored in the compact L-BFGS formula. On exit col is unchanged.
<i>v</i>	On entry v specifies vector v. On exit v is unchanged.
<i>p</i>	On entry p is unspecified. On exit p is the product Mv.
<i>info</i>	On entry info is unspecified. On exit info is always 0. (The original LINPACK dtrsl could set info nonzero on a singular triangular system; the LAPACK dtrsm replacement does not return such a status, so bmv now sets info = 0 unconditionally and callers checking "if info /= 0 ... return" hit dead code.)

Definition at line 37 of file bmv.f.

Referenced by `cauchy()`, and `cmprlb()`.

Here is the caller graph for this function:



3.3 src/cauchy.f File Reference

Functions/Subroutines

- subroutine `cauchy` (n, x, l, u, nbd, g, iorder, iwhere, t, d, xcp, m, wy, ws, sy, wt, theta, col, head, p, c, wbp, v, nseg, iprint, sbgnrm, info, epsmch)

Compute the Generalized Cauchy Point along the projected gradient direction.

3.3.1 Function/Subroutine Documentation

3.3.1.1 cauchy() subroutine cauchy (
integer *n*,
double precision, dimension(*n*) *x*,
double precision, dimension(*n*) *l*,
double precision, dimension(*n*) *u*,
integer, dimension(*n*) *nbd*,
double precision, dimension(*n*) *g*,
integer, dimension(*n*) *iorder*,
integer, dimension(*n*) *iwhere*,
double precision, dimension(*n*) *t*,
double precision, dimension(*n*) *d*,
double precision, dimension(*n*) *xcp*,
integer *m*,
double precision, dimension(*n*, col) *wy*,
double precision, dimension(*n*, col) *ws*,
double precision, dimension(*m*, *m*) *sy*,
double precision, dimension(*m*, *m*) *wt*,
double precision *theta*,
integer *col*,
integer *head*,
double precision, dimension(2**m*) *p*,
double precision, dimension(2**m*) *c*,
double precision, dimension(2**m*) *wbp*,
double precision, dimension(2**m*) *v*,
integer *nseg*,
integer *iprint*,
double precision *sbgnrm*,
integer *info*,
double precision *epsmch*)

For given *x*, *l*, *u*, *g* (with *sbgnrm* > 0), and a limited memory BFGS matrix *B* defined in terms of matrices *WY*, *WS*, *WT*, and scalars *head*, *col*, and *theta*, this subroutine computes the generalized Cauchy point (GCP), defined as the first local minimizer of the quadratic

$$Q(x + s) = g's + 1/2 s'Bs$$

along the projected gradient direction $P(x-tg, l, u)$. The routine returns the GCP in *xcp*.

Parameters

<i>n</i>	On entry <i>n</i> is the dimension of the problem. On exit <i>n</i> is unchanged.
<i>x</i>	On entry <i>x</i> is the starting point for the GCP computation. On exit <i>x</i> is unchanged.
<i>l</i>	On entry <i>l</i> is the lower bound of <i>x</i> . On exit <i>l</i> is unchanged.
<i>u</i>	On entry <i>u</i> is the upper bound of <i>x</i> . On exit <i>u</i> is unchanged.

Parameters

<i>nbd</i>	On entry <i>nbd</i> represents the type of bounds imposed on the variables, and must be specified as follows: $nbd(i)=$ • 0 if $x(i)$ is unbounded, • 1 if $x(i)$ has only a lower bound, • 2 if $x(i)$ has both lower and upper bounds, and • 3 if $x(i)$ has only an upper bound. On exit <i>nbd</i> is unchanged.
<i>g</i>	On entry <i>g</i> is the gradient of $f(x)$. <i>g</i> must be a nonzero vector. On exit <i>g</i> is unchanged.
<i>iorder</i>	<i>iorder</i> will be used to store the breakpoints in the piecewise linear path and free variables encountered. On exit, • $iorder(1), \dots, iorder(nleft)$ are indices of breakpoints which have not been encountered; • $iorder(nleft+1), \dots, iorder(nbreak)$ are indices of encountered breakpoints; and • $iorder(nfree), \dots, iorder(n)$ are indices of variables which have no bound constraints along the search direction.
<i>iwhere</i>	On entry <i>iwhere</i> indicates only the permanently fixed ($iwhere=3$) or free ($iwhere=-1$) components of x. On exit <i>iwhere</i> records the status of the current x variables. $iwhere(i)=$ • -3 if $x(i)$ is free and has bounds, but is not moved • 0 if $x(i)$ is free and has bounds, and is moved • 1 if $x(i)$ is fixed at $l(i)$, and $l(i) \neq u(i)$ • 2 if $x(i)$ is fixed at $u(i)$, and $u(i) \neq l(i)$ • 3 if $x(i)$ is always fixed, i.e., $u(i)=x(i)=l(i)$ • -1 if $x(i)$ is always free, i.e., it has no bounds.
<i>t</i>	working array; will be used to store the break points.
<i>d</i>	the Cauchy direction $P(x-tg)-x$
<i>xcp</i>	returns the GCP on exit
<i>m</i>	On entry <i>m</i> is the maximum number of variable metric corrections used to define the limited memory matrix. On exit <i>m</i> is unchanged.
<i>ws</i>	On entry this stores <i>S</i>, a set of <i>s</i>-vectors, that defines the limited memory BFGS matrix. On exit this array is unchanged.
<i>wy</i>	On entry this stores <i>Y</i>, a set of <i>y</i>-vectors, that defines the limited memory BFGS matrix. On exit this array is unchanged.
<i>sy</i>	On entry this stores $S^T Y$, that defines the limited memory BFGS matrix. On exit this array is unchanged.
<i>wt</i>	On entry this stores the Cholesky factorization of $(\theta * S^T S + LD^{(-1)} L^T)$, that defines the limited memory BFGS matrix. On exit this array is unchanged.
<i>theta</i>	On entry <i>theta</i> is the scaling factor specifying $B_0 = \theta I$. On exit <i>theta</i> is unchanged.

Parameters

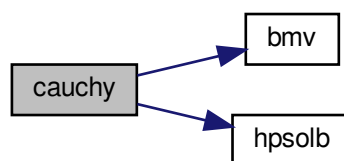
<i>col</i>	On entry col is the actual number of variable metric corrections stored so far. On exit col is unchanged.
<i>head</i>	On entry head is the location of the first s-vector (or y-vector) in S (or Y). On exit col is unchanged.
<i>p</i>	will be used to store the vector $p = W^{\wedge}(T)d$.
<i>c</i>	will be used to store the vector $c = W^{\wedge}(T)(xcp-x)$.
<i>wbp</i>	will be used to store the row of W corresponding to a breakpoint.
<i>v</i>	working array
<i>nseg</i>	On exit nseg records the number of quadratic segments explored in searching for the GCP.
<i>iprint</i>	variable that must be set by the user. It controls the frequency and type of output generated: • $iprint < 0$ no output is generated; • $iprint = 0$ print only one line at the last iteration; • $0 < iprint < 99$ print also f and $proj\ g$ every $iprint$ iterations; • $iprint = 99$ print details of every iteration except n-vectors; • $iprint = 100$ print also the changes of active set and final x; • $iprint > 100$ print details of every iteration including x and g; When $iprint > 0$, the file iterate.dat will be created to summarize the iteration.
<i>sbgnrm</i>	On entry sbgnrm is the norm of the projected gradient at x. On exit sbgnrm is unchanged.
<i>info</i>	On entry info is 0. On exit info = • 0 for normal return, • = nonzero for abnormal return when the the system used in routine bmv is singular.
<i>epsmch</i>	machine precision epsilon

Definition at line 130 of file `cauchy.f`.

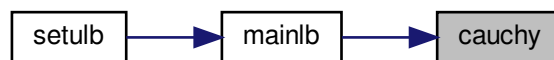
References `bmv()`, and `hpsolb()`.

Referenced by `mainlb()`.

Here is the call graph for this function:



Here is the caller graph for this function:



3.4 src/cmprlb.f File Reference

Functions/Subroutines

- subroutine [cmprlb](#) (*n*, *m*, *x*, *g*, *ws*, *wy*, *sy*, *wt*, *z*, *r*, *wa*, *index*, *theta*, *col*, *head*, *nfree*, *cnstnd*, *info*)
*This subroutine computes $r = -Z'B(xcp-xk) - Z'g$ by using $wa(2m+1) = W'(xcp-x)$ from subroutine *cauchy*.*

3.4.1 Function/Subroutine Documentation

3.4.1.1 cmprlb() `subroutine cmprlb (`
 `integer n,`
 `integer m,`
 `double precision, dimension(n) x,`
 `double precision, dimension(n) g,`
 `double precision, dimension(n, m) ws,`
 `double precision, dimension(n, m) wy,`
 `double precision, dimension(m, m) sy,`
 `double precision, dimension(m, m) wt,`
 `double precision, dimension(n) z,`
 `double precision, dimension(n) r,`
 `double precision, dimension(4*m) wa,`
 `integer, dimension(n) index,`
 `double precision theta,`
 `integer col,`
 `integer head,`
 `integer nfree,`
 `logical cnstnd,`
 `integer info )`

This subroutine computes $r = -Z'B(xcp-xk) - Z'g$ by using $wa(2m+1) = W'(xcp-x)$ from subroutine *cauchy*.

Parameters

<i>n</i>	number of parameters
<i>m</i>	history size of Hessian approximation
<i>x</i>	position
<i>g</i>	gradient
<i>ws</i>	part of L-BFGS matrix

Parameters

<i>wy</i>	part of L-BFGS matrix
<i>sy</i>	part of L-BFGS matrix
<i>wt</i>	part of L-BFGS matrix
<i>z</i>	The generalized Cauchy point xcp computed by cauchy. Used here as the linearisation point: r encodes $-B(z-x) - g$ restricted to the free variables.
<i>r</i>	On exit $r(1:nfree)$ contains $-\theta(z(k)-x(k)) - g(k)$ plus the $W \cdot M^{-1} \cdot W'$ correction, where $k = \text{index}(i)$. Caller uses this as the residual for the subspace minimisation problem. When <i>cnstnd</i> =.false. and <i>col</i> >0, the shortcut path sets $r(1:n) = -g(:)$ without using the index array.
<i>wa</i>	Length-4m workspace shared with cauchy. On entry, the segment $wa(2m+1 : 2m+2col)$ holds $W'(z-x)$ (filled by cauchy). On exit $wa(1 : 2col)$ holds $M^{-1} \cdot W'(z-x)$ from the bmv call.
<i>index</i>	Permutation of $(1..n)$: $\text{index}(1..nfree)$ lists the indices of variables that are free at the GCP and are the active optimisation variables here.
<i>theta</i>	Scaling factor specifying the initial Hessian $B_0 = \theta \cdot I$.
<i>col</i>	Number of stored (s,y) correction pairs (0 on the first iteration; up to m thereafter).
<i>head</i>	Index in the cyclic WS/WY buffer of the oldest stored correction. Used to walk the columns in chronological order.
<i>nfree</i>	Number of free variables; size of the subspace problem.
<i>cnstnd</i>	.true. if the problem has bounds; controls the shortcut path described under
<i>r.</i>	
<i>info</i>	Error code from the embedded bmv call. With the current dtrsm-based bmv this is always 0 (see bmv.f); the code path that maps a nonzero info to -8 is dead but kept for the event of a future BLAS substitution.

Definition at line 43 of file cmprlb.f.

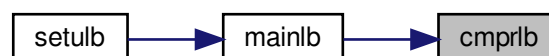
References [bmv\(\)](#).

Referenced by [mainlb\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.5 src/dcsrch.f File Reference

Functions/Subroutines

- subroutine `dcsrch` (`f`, `g`, `stp`, `ftol`, `gtol`, `xtol`, `stpmin`, `stpmax`, `task`, `isave`, `dsave`)

This subroutine finds a step that satisfies a sufficient decrease condition and a curvature condition.

3.5.1 Function/Subroutine Documentation

3.5.1.1 dcsrch() `subroutine dcsrch (`
`double precision f,`
`double precision g,`
`double precision stp,`
`double precision ftol,`
`double precision gtol,`
`double precision xtol,`
`double precision stpmin,`
`double precision stpmax,`
`character*(*) task,`
`integer, dimension(2) isave,`
`double precision, dimension(13) dsave )`

This subroutine finds a step that satisfies a sufficient decrease condition and a curvature condition.

Each call of the subroutine updates an interval with endpoints `stx` and `sty`. The interval is initially chosen so that it contains a minimizer of the modified function

$$\text{psi}(\text{stp}) = f(\text{stp}) - f(0) - \text{ftol} \cdot \text{stp} \cdot f'(0).$$

If $\text{psi}(\text{stp}) \leq 0$ and $f'(\text{stp}) \geq 0$ for some step, then the interval is chosen so that it contains a minimizer of f .

The algorithm is designed to find a step that satisfies the sufficient decrease condition

$$f(\text{stp}) \leq f(0) + \text{ftol} \cdot \text{stp} \cdot f'(0),$$

and the curvature condition

$$\text{abs}(f'(\text{stp})) \leq \text{gtol} \cdot \text{abs}(f'(0)).$$

If `ftol` is less than `gtol` and if, for example, the function is bounded below, then there is always a step which satisfies both conditions.

If no step can be found that satisfies both conditions, then the algorithm stops with a warning. In this case `stp` only satisfies the sufficient decrease condition.

A typical invocation of `dcsrch` has the following outline:

```
task = 'START'
10 continue
call dcsrch( ... )
if (task .eq. 'FG') then
    evaluate the function and the gradient at stp
goto 10
end if
```

NOTE: The user must not alter work arrays between calls.

Parameters

<i>f</i>	On initial entry <i>f</i> is the value of the function at 0. On subsequent entries <i>f</i> is the value of the function at <i>stp</i> . On exit <i>f</i> is the value of the function at <i>stp</i> .
<i>g</i>	On initial entry <i>g</i> is the derivative of the function at 0. On subsequent entries <i>g</i> is the derivative of the function at <i>stp</i> . On exit <i>g</i> is the derivative of the function at <i>stp</i> .
<i>stp</i>	On entry <i>stp</i> is the current estimate of a satisfactory step. On initial entry, a positive initial estimate must be provided. On exit <i>stp</i> is the current estimate of a satisfactory step if task = 'FG'. If task = 'CONV' then <i>stp</i> satisfies the sufficient decrease and curvature condition.
<i>ftol</i>	On entry <i>ftol</i> specifies a nonnegative tolerance for the sufficient decrease condition. On exit <i>ftol</i> is unchanged.
<i>gtol</i>	On entry <i>gtol</i> specifies a nonnegative tolerance for the curvature condition. On exit <i>gtol</i> is unchanged.
<i>xtol</i>	On entry <i>xtol</i> specifies a nonnegative relative tolerance for an acceptable step. The subroutine exits with a warning if the relative difference between <i>sty</i> and <i>stx</i> is less than <i>xtol</i> . On exit <i>xtol</i> is unchanged.
<i>stpmin</i>	On entry <i>stpmin</i> is a nonnegative lower bound for the step. On exit <i>stpmin</i> is unchanged.
<i>stpmax</i>	On entry <i>stpmax</i> is a nonnegative upper bound for the step. On exit <i>stpmax</i> is unchanged.
<i>task</i>	On initial entry <i>task</i> must be set to 'START'. On exit <i>task</i> indicates the required action: • If <i>task</i>(1:2) = 'FG' then evaluate the function and derivative at <i>stp</i> and call <i>dcsrch</i> again. • If <i>task</i>(1:4) = 'CONV' then the search is successful. • If <i>task</i>(1:4) = 'WARN' then the subroutine is not able to satisfy the convergence conditions. The exit value of <i>stp</i> contains the best point found during the search. • If <i>task</i>(1:5) = 'ERROR' then there is an error in the input arguments. On exit with convergence, a warning or an error, the variable <i>task</i> contains additional information.
<i>isave</i>	work array
<i>dsave</i>	work array

Definition at line 94 of file *dcsrch.f*.

References *dcstep*().

Referenced by *Insrlb*().

Here is the call graph for this function:



Here is the caller graph for this function:



3.6 src/dcstep.f File Reference

Functions/Subroutines

- subroutine [dcstep](#) (stx, fx, dx, sty, fy, dy, stp, fp, dp, brackt, stpmin, stpmax)

This subroutine computes a safeguarded step for a search procedure and updates an interval that contains a step that satisfies a sufficient decrease and a curvature condition.

3.6.1 Function/Subroutine Documentation

3.6.1.1 dcstep() `subroutine dcstep (`
 double precision *stx*,
 double precision *fx*,
 double precision *dx*,
 double precision *sty*,
 double precision *fy*,
 double precision *dy*,
 double precision *stp*,
 double precision *fp*,
 double precision *dp*,
 logical *brackt*,
 double precision *stpmin*,
 double precision *stpmax*)

This subroutine computes a safeguarded step for a search procedure and updates an interval that contains a step that satisfies a sufficient decrease and a curvature condition.

The parameter *stx* contains the step with the least function value. If *brackt* is set to `.true.` then a minimizer has been bracketed in an interval with endpoints *stx* and *sty*. The parameter *stp* contains the current step. The subroutine assumes that if *brackt* is set to `.true.` then

```
min(stx, sty) < stp < max(stx, sty),
```

and that the derivative at *stx* is negative in the direction of the step.

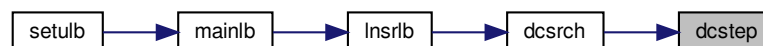
Parameters

<i>stx</i>	On entry <i>stx</i> is the best step obtained so far and is an endpoint of the interval that contains the minimizer. On exit <i>stx</i> is the updated best step.
<i>fx</i>	On entry <i>fx</i> is the function at <i>stx</i> . On exit <i>fx</i> is the function at <i>stx</i> .
<i>dx</i>	On entry <i>dx</i> is the derivative of the function at <i>stx</i> . The derivative must be negative in the direction of the step, that is, <i>dx</i> and <i>stp - stx</i> must have opposite signs. On exit <i>dx</i> is the derivative of the function at <i>stx</i> .
<i>sty</i>	On entry <i>sty</i> is the second endpoint of the interval that contains the minimizer. On exit <i>sty</i> is the updated endpoint of the interval that contains the minimizer.
<i>fy</i>	On entry <i>fy</i> is the function at <i>sty</i> . On exit <i>fy</i> is the function at <i>sty</i> .
<i>dy</i>	On entry <i>dy</i> is the derivative of the function at <i>sty</i> . On exit <i>dy</i> is the derivative of the function at the exit <i>sty</i> .
<i>stp</i>	On entry <i>stp</i> is the current step. If <i>brackt</i> is set to <i>.true.</i> then on input <i>stp</i> must be between <i>stx</i> and <i>sty</i> . On exit <i>stp</i> is a new trial step.
<i>fp</i>	On entry <i>fp</i> is the function at <i>stp</i> . On exit <i>fp</i> is unchanged.
<i>dp</i>	On entry <i>dp</i> is the the derivative of the function at <i>stp</i> . On exit <i>dp</i> is unchanged.
<i>brackt</i>	On entry <i>brackt</i> specifies if a minimizer has been bracketed. Initially <i>brackt</i> must be set to <i>.false.</i> On exit <i>brackt</i> specifies if a minimizer has been bracketed. When a minimizer is bracketed <i>brackt</i> is set to <i>.true.</i>
<i>stpmin</i>	On entry <i>stpmin</i> is a lower bound for the step. On exit <i>stpmin</i> is unchanged.
<i>stpmax</i>	On entry <i>stpmax</i> is an upper bound for the step. On exit <i>stpmax</i> is unchanged.

Definition at line 66 of file *dcstep.f*.

Referenced by *dcsrch()*.

Here is the caller graph for this function:



3.7 src/errclb.f File Reference

Functions/Subroutines

- subroutine [errclb](#) (*n*, *m*, *factr*, *l*, *u*, *nbd*, *task*, *info*, *k*)

This subroutine checks the validity of the input data.

3.7.1 Function/Subroutine Documentation

3.7.1.1 errclb() subroutine errclb (
integer *n*,
integer *m*,
double precision *factr*,
double precision, dimension(*n*) *l*,
double precision, dimension(*n*) *u*,
integer, dimension(*n*) *nbd*,
character*60 *task*,
integer *info*,
integer *k*)

This subroutine checks the validity of the input data.

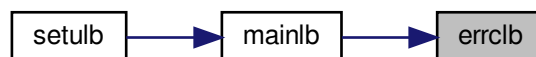
Parameters

<i>n</i>	number of parameters
<i>m</i>	history size of approximated Hessian
<i>factr</i>	convergence criterion on function value
<i>l</i>	lower bounds for parameters
<i>u</i>	upper bounds for parameters
<i>nbd</i>	indicates which bounds are present
<i>task</i>	if an error occurs, contains a human-readable error message
<i>info</i>	=0 on success; =-6 if <i>nbd</i> (<i>k</i>) was invalid; =-7 if both limits are given but $l(k) > u(k)$
<i>k</i>	index of last erroneous parameter

Definition at line 16 of file errclb.f.

Referenced by mainlb().

Here is the caller graph for this function:



3.8 src/formk.f File Reference

Functions/Subroutines

- subroutine [formk](#) (*n*, *nsub*, *ind*, *nenter*, *ileave*, *indx2*, *iupdat*, *updatd*, *wn*, *wn1*, *m*, *ws*, *wy*, *sy*, *theta*, *col*, *head*, *info*)

Forms the LEL^T factorization of the indefinite matrix K .

3.8.1 Function/Subroutine Documentation

3.8.1.1 formk() subroutine formk (

```

    integer n,
    integer nsub,
    integer, dimension(n) ind,
    integer nenter,
    integer ileave,
    integer, dimension(n) indx2,
    integer iupdat,
    logical updatd,
    double precision, dimension(2*m, 2*m) wn,
    double precision, dimension(2*m, 2*m) wn1,
    integer m,
    double precision, dimension(n, m) ws,
    double precision, dimension(n, m) wy,
    double precision, dimension(m, m) sy,
    double precision theta,
    integer col,
    integer head,
    integer info )

```

This subroutine forms the LEL^T factorization of the indefinite matrix

$$K = [-D \ -Y'ZZ'Y/\theta \ L_a' -R_z'] [L_a \ -R_z \ \theta S'AA'S] \text{ where } E = [-I \ 0] [0 \ I]$$

The matrix K can be shown to be equal to the matrix $M^{-1}N$ occurring in section 5.1 of [1], as well as to the matrix $Mbar^{-1}Nbar$ in section 5.3.

Parameters

<i>n</i>	On entry n is the dimension of the problem. On exit n is unchanged.
<i>nsub</i>	On entry nsub is the number of subspace variables in free set. On exit nsub is not changed.
<i>ind</i>	On entry ind specifies the indices of subspace variables. On exit ind is unchanged.
<i>nenter</i>	On entry nenter is the number of variables entering the free set. On exit nenter is unchanged.
<i>ileave</i>	On entry indx2(ileave),...,indx2(n) are the variables leaving the free set. On exit ileave is unchanged.
<i>indx2</i>	On entry indx2(1),...,indx2(nenter) are the variables entering the free set, while indx2(ileave),...,indx2(n) are the variables leaving the free set. On exit indx2 is unchanged.
<i>iupdat</i>	On entry iupdat is the total number of BFGS updates made so far. On exit iupdat is unchanged.
<i>updatd</i>	On entry 'updatd' is true if the L-BFGS matrix is updated. On exit 'updatd' is unchanged.
<i>wn</i>	On entry wn is unspecified. On exit the upper triangle of wn stores the LEL^T factorization of the $2*col \times 2*col$ indefinite matrix [-D -Y'ZZ'Y/theta L_a' -R_z'] [L_a -R_z theta*S'AA'S]

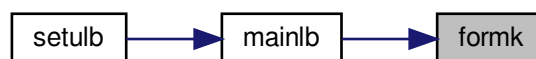
Parameters

<i>wn1</i>	On entry wn1 stores the lower triangular part of $[Y' ZZ'Y L_a' + R_z'] [L_a + R_z S'AA'S]$ in the previous iteration. On exit wn1 stores the corresponding updated matrices. The purpose of wn1 is just to store these inner products so they can be easily updated and inserted into wn.
<i>m</i>	On entry m is the maximum number of variable metric corrections used to define the limited memory matrix. On exit m is unchanged.
<i>ws</i>	On entry this stores S, a set of s-vectors, that defines the limited memory BFGS matrix. On exit this array is unchanged.
<i>wy</i>	On entry this stores Y, a set of y-vectors, that defines the limited memory BFGS matrix. On exit this array is unchanged.
<i>sy</i>	On entry this stores $S'Y$, that defines the limited memory BFGS matrix. On exit this array is unchanged.
<i>theta</i>	On entry theta is the scaling factor specifying $B_0 = \theta I$. On exit theta is unchanged.
<i>col</i>	On entry col is the actual number of variable metric corrections stored so far. On exit col is unchanged.
<i>head</i>	On entry head is the location of the first s-vector (or y-vector) in S (or Y). On exit col is unchanged.
<i>info</i>	On entry info is unspecified. On exit info = 0 for normal return; = -1 when the 1st Cholesky factorization failed; = -2 when the 2st Cholesky factorization failed.

Definition at line 89 of file formk.f.

Referenced by mainlb().

Here is the caller graph for this function:



3.9 src/formt.f File Reference

Functions/Subroutines

- subroutine [formt](#) (m, wt, sy, ss, col, theta, info)
Forms the upper half of the pos. def. and symm. T.

3.9.1 Function/Subroutine Documentation

3.9.1.1 `formt()` `subroutine formt (`
`integer m,`
`double precision, dimension(m, m) wt,`
`double precision, dimension(m, m) sy,`
`double precision, dimension(m, m) ss,`
`integer col,`
`double precision theta,`
`integer info )`

This subroutine forms the upper half of the pos. def. and symm. $T = \theta * SS + L * D^{(-1)} * L'$, stores T in the upper triangle of the array wt, and performs the Cholesky factorization of T to produce $J * J'$, with J' stored in the upper triangle of wt.

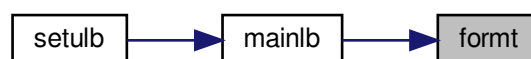
Parameters

<i>m</i>	history size of approximated Hessian
<i>wt</i>	part of L-BFGS matrix
<i>sy</i>	part of L-BFGS matrix
<i>ss</i>	part of L-BFGS matrix
<i>col</i>	On entry col is the actual number of variable metric corrections stored so far. On exit col is unchanged.
<i>theta</i>	On entry theta is the scaling factor specifying $B_0 = \theta I$. On exit theta is unchanged.
<i>info</i>	error/success indicator

Definition at line 21 of file formt.f.

Referenced by mainlb().

Here is the caller graph for this function:



3.10 src/freev.f File Reference

Functions/Subroutines

- subroutine [freev](#) (n, nfree, index, nenter, ileave, indx2, iwhere, wrk, updatd, cnstnd, iprint, iter)

This subroutine counts the entering and leaving variables when iter > 0, and finds the index set of free and active variables at the GCP.

3.10.1 Function/Subroutine Documentation

3.10.1.1 freev() subroutine freev (
 integer *n*,
 integer *nfree*,
 integer, dimension(*n*) *index*,
 integer *nenter*,
 integer *ileave*,
 integer, dimension(*n*) *indx2*,
 integer, dimension(*n*) *iwhere*,
 logical *wrk*,
 logical *updatd*,
 logical *cnstnd*,
 integer *iprint*,
 integer *iter*)

This subroutine counts the entering and leaving variables when *iter* > 0, and finds the index set of free and active variables at the GCP.

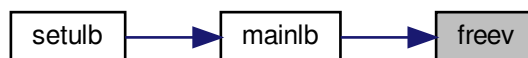
Parameters

<i>n</i>	number of parameters
<i>nfree</i>	number of free parameters, i.e., those not at their bounds
<i>index</i>	for <i>i</i> =1,..., <i>nfree</i> , <i>index</i> (<i>i</i>) are the indices of free variables for <i>i</i> = <i>nfree</i> +1,..., <i>n</i> , <i>index</i> (<i>i</i>) are the indices of bound variables On entry after the first iteration, <i>index</i> gives the free variables at the previous iteration. On exit it gives the free variables based on the determination in cauchy using the array <i>iwhere</i> .
<i>nenter</i>	On exit <i>nenter</i> is the number of variables that entered the free set this iteration (were active, now free at the GCP).
<i>ileave</i>	On exit <i>indx2</i> (<i>ileave</i>),..., <i>indx2</i> (<i>n</i>) list the variables that left the free set this iteration. <i>ileave</i> starts at <i>n</i> +1 and is decremented each time a leaving variable is recorded.
<i>indx2</i>	On entry <i>indx2</i> is unspecified. On exit with <i>iter</i> >0, <i>indx2</i> indicates which variables have changed status since the previous iteration. For <i>i</i> = 1,..., <i>nenter</i> , <i>indx2</i> (<i>i</i>) have changed from bound to free. For <i>i</i> = <i>ileave</i> +1,..., <i>n</i> , <i>indx2</i> (<i>i</i>) have changed from free to bound.
<i>iwhere</i>	On entry <i>iwhere</i> (<i>i</i>) classifies each variable's bound status (set by cauchy): ≤0 means free at GCP, >0 means at-bound. Used here to compare against the previous <i>index</i> / <i>nfree</i> to detect leaving and entering variables.
<i>wrk</i>	On exit .true. if the active-set or L-BFGS bookkeeping has changed enough that the workspace <i>WN</i> needs to be rebuilt (some variable entered/left, or <i>updatd</i> is .true.).
<i>updatd</i>	On entry .true. if the L-BFGS matrix was updated in the previous iteration. Combined with the entering/leaving counts to set <i>wrk</i> .
<i>cnstnd</i>	Whether bounds are present (true if at least one variable is bounded). When false, the entering/leaving counting loop is skipped.
<i>iprint</i>	Console output flag (≥99 prints summary, ≥100 prints per-variable change records).
<i>iter</i>	Current outer iteration number. The entering/leaving counting loop only runs when <i>iter</i> > 0 (the first iteration has no "previous" set to compare against).

Definition at line 47 of file *freev.f*.

Referenced by *mainlb*().

Here is the caller graph for this function:



3.11 src/hpsolb.f File Reference

Functions/Subroutines

- subroutine [hpsolb](#) (*n*, *t*, *iorder*, *iheap*)

*This subroutine sorts out the least element of *t*, and puts the remaining elements of *t* in a heap.*

3.11.1 Function/Subroutine Documentation

3.11.1.1 hpsolb() `subroutine hpsolb (`
 `integer n,`
 `double precision, dimension(n) t,`
 `integer, dimension(n) iorder,`
 `integer iheap )`

Parameters

<i>n</i>	On entry <i>n</i> is the dimension of the arrays <i>t</i> and <i>iorder</i> . On exit <i>n</i> is unchanged.
<i>t</i>	On entry <i>t</i> stores the elements to be sorted. On exit <i>t</i> (<i>n</i>) stores the least elements of <i>t</i> , and <i>t</i> (1) to <i>t</i> (<i>n</i> -1) stores the remaining elements in the form of a heap.
<i>iorder</i>	On entry <i>iorder</i> (<i>i</i>) is the index of <i>t</i> (<i>i</i>). On exit <i>iorder</i> (<i>i</i>) is still the index of <i>t</i> (<i>i</i>), but <i>iorder</i> may be permuted in accordance with <i>t</i> .
<i>iheap</i>	On entry <i>iheap</i> should be set as follows: <i>iheap</i> .eq. 0 if <i>t</i>(1) to <i>t</i>(<i>n</i>) is not in the form of a heap, <i>iheap</i> .ne. 0 if otherwise. On exit <i>iheap</i> is unchanged.

Definition at line 21 of file hpsolb.f.

Referenced by [cauchy\(\)](#).

Here is the caller graph for this function:



3.12 src/lnsrlb.f File Reference

Functions/Subroutines

- subroutine [lnsrlb](#) (n, l, u, nbd, x, f, fold, gd, gdold, g, d, r, t, z, stp, dnorm, dtd, xstep, stpmx, iter, ifun, iback, nfgv, info, task, boxed, cnstnd, csave, isave, dsave)

This subroutine calls subroutine dscrch from the Minpack2 library to perform the line search. Subroutine dscrch is safeguarded so that all trial points lie within the feasible region.

3.12.1 Function/Subroutine Documentation

3.12.1.1 lnsrlb() `subroutine lnsrlb (`
 integer *n*,
 double precision, dimension(*n*) *l*,
 double precision, dimension(*n*) *u*,
 integer, dimension(*n*) *nbd*,
 double precision, dimension(*n*) *x*,
 double precision *f*,
 double precision *fold*,
 double precision *gd*,
 double precision *gdold*,
 double precision, dimension(*n*) *g*,
 double precision, dimension(*n*) *d*,
 double precision, dimension(*n*) *r*,
 double precision, dimension(*n*) *t*,
 double precision, dimension(*n*) *z*,
 double precision *stp*,
 double precision *dnorm*,
 double precision *dtd*,
 double precision *xstep*,
 double precision *stpmx*,
 integer *iter*,
 integer *ifun*,
 integer *iback*,
 integer *nfgv*,
 integer *info*,
 character*60 *task*,
 logical *boxed*,
 logical *cnstnd*,
 character*60 *csave*,
 integer, dimension(2) *isave*,
 double precision, dimension(13) *dsave*)

Parameters

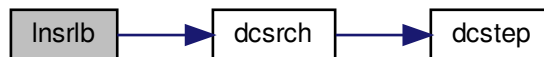
<i>n</i>	number of parameters
<i>l</i>	lower bounds of parameters
<i>u</i>	upper bounds of parameters
<i>nbd</i>	On entry <i>nbd</i> represents the type of bounds imposed on the variables, and must be specified as follows: <i>nbd</i> (<i>i</i>)= • 0 if <i>x</i>(<i>i</i>) is unbounded, • 1 if <i>x</i>(<i>i</i>) has only a lower bound, • 2 if <i>x</i>(<i>i</i>) has both lower and upper bounds, and • 3 if <i>x</i>(<i>i</i>) has only an upper bound. On exit <i>nbd</i> is unchanged.
<i>x</i>	position
<i>f</i>	function value at <i>x</i>
<i>fold</i>	Function value at the start of this line search (i.e. the accepted value from the previous iteration). Saved at entry so the caller can restore <i>x</i> , <i>g</i> , <i>f</i> if the line search fails.
<i>gd</i>	Directional derivative <i>g</i> ' <i>d</i> at the current trial step. Computed on every entry and passed to <i>dcsrch</i> as its <i>g</i> argument.
<i>gdold</i>	Directional derivative at <i>stp</i> =0 (i.e. the initial <i>g</i> ' <i>d</i> before any line-search progress this iteration). Saved on the first call and used by <i>mainlb</i> to test the curvature condition after the line search returns.
<i>g</i>	Gradient of <i>f</i> at <i>x</i> .
<i>d</i>	Search direction (<i>z</i> - <i>x</i> _{current}). Length- <i>n</i> vector; the candidate step is <i>t</i> + <i>stp</i> * <i>d</i> .
<i>r</i>	Workspace: copy of <i>g</i> at the start of this line search. Used alongside <i>fold</i> to restore the previous iterate on abnormal line-search termination (<i>mainlb</i> does the restore from <i>r</i> and <i>t</i>).
<i>t</i>	Workspace: copy of <i>x</i> at the start of this line search.
<i>z</i>	Pre-projected candidate (cauchy/subsm output). When <i>stp</i> =1 exactly, <i>Insr1b</i> sets <i>x</i> := <i>z</i> directly; otherwise it computes <i>x</i> := <i>t</i> + <i>stp</i> * <i>d</i> .
<i>stp</i>	Current trial step length. On the first entry of a line search <i>Insr1b</i> initialises <i>stp</i> ; subsequent <i>dcsrch</i> calls update it.
<i>dnorm</i>	2-norm of <i>d</i> ($\ d\ $).
<i>dtd</i>	Squared 2-norm of <i>d</i> (<i>d</i> ' <i>d</i>).
<i>xstep</i>	On exit <i>stp</i> * $\ d\ $, the actual length of the step in <i>x</i> -space.
<i>stpmx</i>	Maximum allowed step. For unconstrained problems set to a large constant (1e10); for bounded problems <i>Insr1b</i> scans the active bounds and tightens <i>stpmx</i> so <i>x</i> + <i>stpmx</i> * <i>d</i> stays feasible.
<i>iter</i>	Outer iteration number from <i>mainlb</i> .
<i>ifun</i>	On exit number of <i>f</i> / <i>g</i> evaluations performed in this line search; reset to 0 on each new line search.
<i>iback</i>	On exit number of "backtracks" (<i>ifun</i> - 1). <i>mainlb</i> aborts the line search if <i>iback</i> ≥ 20.
<i>nfgv</i>	Cumulative count of <i>f</i> / <i>g</i> evaluations across all iterations; incremented by 1 per evaluation requested.
<i>info</i>	On exit 0 on success; -4 if the projected directional derivative <i>gd</i> is non-negative on the first call (no descent possible).
<i>task</i>	Reverse-comm task. Initial entry: 'START'. While the line search is running, <i>Insr1b</i> returns 'FG_LNSRCH' (the user evaluates <i>f</i> , <i>g</i> at the new <i>x</i> and re-enters with <i>task</i> starting with 'FG_LN'). On line-search success <i>Insr1b</i> returns 'NEW_X'.
<i>boxed</i>	.true. if every variable has both lower and upper bounds. When true, the initial trial step is unit (<i>stp</i> =1) regardless of <i>d</i> 's magnitude.
<i>cnstnd</i>	.true. if the problem has at least one bound. Controls the <i>stpmx</i> -from-bounds scan.
<i>csave</i>	working array
<i>isave</i>	working array
<i>dsave</i>	working array

Definition at line 70 of file `Insrlb.f`.

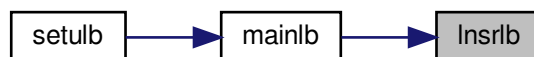
References `dcsrch()`.

Referenced by `mainlb()`.

Here is the call graph for this function:



Here is the caller graph for this function:



3.13 `src/mainlb.f` File Reference

Functions/Subroutines

- subroutine [mainlb](#) (`n`, `m`, `x`, `l`, `u`, `nbd`, `f`, `g`, `factr`, `pgtol`, `ws`, `wy`, `sy`, `ss`, `wt`, `wn`, `snd`, `z`, `r`, `d`, `t`, `xp`, `wa`, `index`, `iwhere`, `indx2`, `task`, `iprint`, `csave`, `lsave`, `isave`, `dsave`)

This subroutine solves bound constrained optimization problems by using the compact formula of the limited memory BFGS updates.

3.13.1 Function/Subroutine Documentation

```

3.13.1.1 mainlb()  subroutine mainlb (
    integer n,
    integer m,
    double precision, dimension(n) x,
    double precision, dimension(n) l,
    double precision, dimension(n) u,
    integer, dimension(n) nbd,
    double precision f,
    double precision, dimension(n) g,
    double precision factr,
    double precision pgtol,
    double precision, dimension(n, m) ws,
    double precision, dimension(n, m) wy,
    double precision, dimension(m, m) sy,
    double precision, dimension(m, m) ss,
    double precision, dimension(m, m) wt,
    double precision, dimension(2*m, 2*m) wn,
    double precision, dimension(2*m, 2*m) snd,
    double precision, dimension(n) z,
    double precision, dimension(n) r,
    double precision, dimension(n) d,
    double precision, dimension(n) t,
    double precision, dimension(n) xp,
    double precision, dimension(8*m) wa,
    integer, dimension(n) index,
    integer, dimension(n) iwhere,
    integer, dimension(n) indx2,
    character*60 task,
    integer iprint,
    character*60 csave,
    logical, dimension(4) lsave,
    integer, dimension(23) isave,
    double precision, dimension(29) dsave )

```

This subroutine solves bound constrained optimization problems by using the compact formula of the limited memory BFGS updates.

Parameters

<i>n</i>	On entry <i>n</i> is the number of variables. On exit <i>n</i> is unchanged.
<i>m</i>	On entry <i>m</i> is the maximum number of variable metric corrections allowed in the limited memory matrix. On exit <i>m</i> is unchanged.
<i>x</i>	On entry <i>x</i> is an approximation to the solution. On exit <i>x</i> is the current approximation.
<i>l</i>	On entry <i>l</i> is the lower bound of <i>x</i> . On exit <i>l</i> is unchanged.
<i>u</i>	On entry <i>u</i> is the upper bound of <i>x</i> . On exit <i>u</i> is unchanged.

Parameters

<i>nbd</i>	On entry <i>nbd</i> represents the type of bounds imposed on the variables, and must be specified as follows: $nbd(i)=$ • 0 if $x(i)$ is unbounded, • 1 if $x(i)$ has only a lower bound, • 2 if $x(i)$ has both lower and upper bounds, • 3 if $x(i)$ has only an upper bound. On exit <i>nbd</i> is unchanged.
<i>f</i>	On first entry <i>f</i> is unspecified. On final exit <i>f</i> is the value of the function at x.
<i>g</i>	On first entry <i>g</i> is unspecified. On final exit <i>g</i> is the value of the gradient at x.
<i>factr</i>	On entry $factr \geq 0$ is specified by the user. The iteration will stop when $(f^k - f^{k+1})/\max\{ f^k , f^{k+1} , 1\} \leq factr \cdot \epsilon_{smch}$ where ϵ_{smch} is the machine precision, which is automatically generated by the code. On exit <i>factr</i> is unchanged.
<i>pgtol</i>	On entry $pgtol \geq 0$ is specified by the user. The iteration will stop when $\max\{ proj\ g_i \mid i = 1, \dots, n\} \leq pgtol$ where pg_i is the ith component of the projected gradient. On exit <i>pgtol</i> is unchanged.
<i>ws</i>	On entry this stores S, a set of s-vectors, that defines the limited memory BFGS matrix. On exit this array is unchanged.
<i>wy</i>	On entry this stores Y, a set of y-vectors, that defines the limited memory BFGS matrix. On exit this array is unchanged.
<i>sy</i>	On entry this stores $S'Y$, that defines the limited memory BFGS matrix. On exit this array is unchanged.
<i>ss</i>	On entry this stores $S'S$, that defines the limited memory BFGS matrix. On exit this array is unchanged.
<i>wt</i>	On entry this stores the Cholesky factorization of $(\theta * S'S + LD^(-1)L')$, that defines the limited memory BFGS matrix. See eq. (2.26) in [3]. On exit this array is unchanged.
<i>wn</i>	working array used to store the LEL^T factorization of the indefinite matrix $K = [-D \ -Y'ZZ'Y/\theta \ L_a' - R_z'] [L_a \ -R_z \ \theta * S'AA'S]$ where $E = [-I \ 0] [0 \ I]$
<i>snd</i>	working array used to store the lower triangular part of $N = [Y' ZZ'Y L_a' + R_z'] [L_a \ +R_z \ S'AA'S]$
<i>z</i>	working array used at different times to store the Cauchy point and the Newton point.
<i>r</i>	working array
<i>d</i>	working array
<i>t</i>	working array
<i>xp</i>	working array used to safeguard the projected Newton direction
<i>wa</i>	working array
<i>index</i>	In subroutine <i>freev</i>, <i>index</i> is used to store the free and fixed variables at the Generalized Cauchy Point (GCP).

Parameters

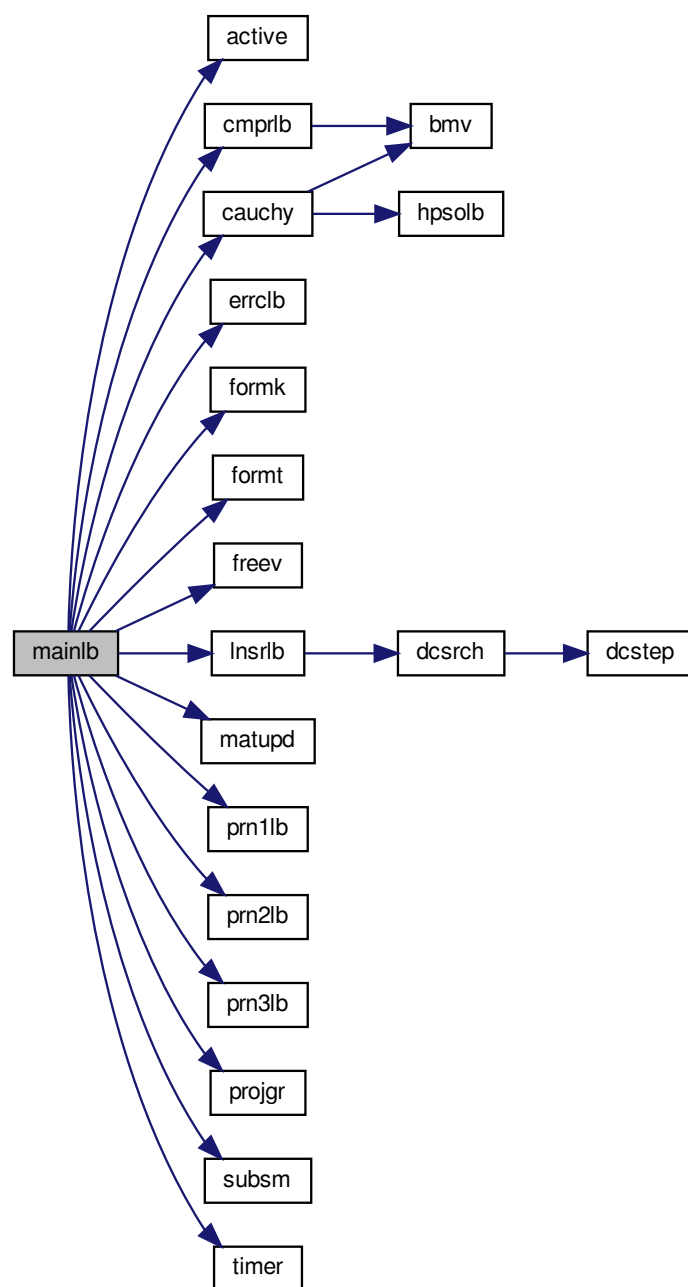
<i>iwhere</i>	working array used to record the status of the vector x for GCP computation. iwhere(i)= • 0 or -3 if x(i) is free and has bounds, • 1 if x(i) is fixed at l(i), and l(i) .ne. u(i) • 2 if x(i) is fixed at u(i), and u(i) .ne. l(i) • 3 if x(i) is always fixed, i.e., u(i)=x(i)=l(i) • -1 if x(i) is always free, i.e., no bounds on it.
<i>indx2</i>	working array Within subroutine cauchy, indx2 corresponds to the array iorder. In subroutine freev, a list of variables entering and leaving the free set is stored in indx2, and it is passed on to subroutine formk with this information.
<i>task</i>	working string indicating the current job when entering and leaving this subroutine.
<i>iprint</i>	It controls the frequency and type of output generated: • iprint<0 no output is generated; • iprint=0 print only one line at the last iteration; • 0<iprint<99 print also f and proj g every iprint iterations; • iprint=99 print details of every iteration except n-vectors; • iprint=100 print also the changes of active set and final x; • iprint>100 print details of every iteration including x and g; When iprint > 0, the file iterate.dat will be created to summarize the iteration.
<i>csave</i>	working string
<i>lsave</i>	working array
<i>isave</i>	working array
<i>dsave</i>	working array

Definition at line 128 of file mainlb.f.

References `active()`, `cauchy()`, `cmprlb()`, `errclb()`, `formk()`, `format()`, `freev()`, `lnsr1b()`, `matupd()`, `prn1lb()`, `prn2lb()`, `prn3lb()`, `projgr()`, `subsm()`, and `timer()`.

Referenced by `setulb()`.

Here is the call graph for this function:



Here is the caller graph for this function:



3.14 src/matupd.f File Reference

Functions/Subroutines

- subroutine [matupd](#) (n, m, ws, wy, sy, ss, d, r, itail, iupdat, col, head, theta, rr, dr, stp, dtd)

This subroutine updates matrices WS and WY, and forms the middle matrix in B.

3.14.1 Function/Subroutine Documentation

3.14.1.1 matupd() subroutine matupd (
 integer *n*,
 integer *m*,
 double precision, dimension(*n*, *m*) *ws*,
 double precision, dimension(*n*, *m*) *wy*,
 double precision, dimension(*m*, *m*) *sy*,
 double precision, dimension(*m*, *m*) *ss*,
 double precision, dimension(*n*) *d*,
 double precision, dimension(*n*) *r*,
 integer *itail*,
 integer *iupdat*,
 integer *col*,
 integer *head*,
 double precision *theta*,
 double precision *rr*,
 double precision *dr*,
 double precision *stp*,
 double precision *dtd*)

This subroutine updates matrices WS and WY, and forms the middle matrix in B.

Parameters

<i>n</i>	On entry <i>n</i> is the number of variables. On exit <i>n</i> is unchanged.
<i>m</i>	On entry <i>m</i> is the maximum number of variable metric corrections allowed in the limited memory matrix. On exit <i>m</i> is unchanged.
<i>ws</i>	On entry this stores S, a set of s-vectors, that defines the limited memory BFGS matrix. On exit this array is unchanged.

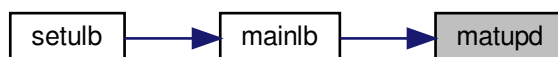
Parameters

<i>wy</i>	On entry this stores Y, a set of y-vectors, that defines the limited memory BFGS matrix. On exit this array is unchanged.
<i>sy</i>	On entry this stores S'Y, that defines the limited memory BFGS matrix. On exit this array is unchanged.
<i>ss</i>	On entry this stores S'S, that defines the limited memory BFGS matrix. On exit this array is unchanged.
<i>d</i>	Search direction at the current iteration. After the line search, the new s-vector is $stp*d$; <i>matupd</i> stores it as a column of WS (writing <i>d</i> directly, since the <i>stp</i> scaling is folded into the stored <i>ss</i> diagonal entry below).
<i>r</i>	The accepted gradient difference $y = g_{\{k+1\}} - g_k$. Stored as a column of WY.
<i>itail</i>	On entry the column index in WS/WY that the previous update wrote. On exit the column index this update wrote; advances cyclically modulo <i>m</i> .
<i>iupdat</i>	Total number of L-BFGS updates performed so far (incremented by <i>mainlb</i> before each <i>matupd</i> call). When <i>iupdat</i> $\leq m$, <i>col</i> simply grows; when <i>iupdat</i> $> m$, the history wraps and the oldest column is discarded.
<i>col</i>	On entry <i>col</i> is the actual number of variable metric corrections stored so far. On exit <i>col</i> is unchanged.
<i>head</i>	On entry <i>head</i> is the location of the first s-vector (or y-vector) in S (or Y). On exit <i>col</i> is unchanged.
<i>theta</i>	On entry <i>theta</i> is the scaling factor specifying $B_0 = \theta I$. On exit <i>theta</i> is unchanged.
<i>rr</i>	Squared 2-norm of <i>r</i> (i.e. $\ y\ ^2 = y'y$). Used to set $\theta := rr/dr = y'y / (s'y)$, the standard initial Hessian scaling for the next L-BFGS update.
<i>dr</i>	Inner product $d'r = s'y / stp$ (the curvature condition; must be positive for the update to be safely accepted – <i>mainlb</i> checks this and skips <i>matupd</i> if $dr \leq \epsilon \cdot rr$).
<i>stp</i>	Line-search step length. Used to recover the s-vector from <i>d</i> ($s = stp*d$) when computing $ss(col,col) = stp^2 * d'd$.
<i>dtd</i>	Squared 2-norm of <i>d</i> (i.e. $d'd$). Combined with <i>stp</i> to form $ss(col,col) = stp^2 * dtd = \ s\ ^2$ for the new column.

Definition at line 64 of file *matupd.f*.

Referenced by *mainlb*().

Here is the caller graph for this function:



3.15 src/prn1lb.f File Reference

Functions/Subroutines

- subroutine [prn1lb](#) (*n*, *m*, *l*, *u*, *x*, *iprint*, *itfile*, *epsmch*)

This subroutine prints the input data, initial point, upper and lower bounds of each variable, machine precision, as well as the headings of the output.

3.15.1 Function/Subroutine Documentation

3.15.1.1 prn1lb() subroutine prn1lb (
integer *n*,
integer *m*,
double precision, dimension(*n*) *l*,
double precision, dimension(*n*) *u*,
double precision, dimension(*n*) *x*,
integer *iprint*,
integer *itfile*,
double precision *epsmch*)

This subroutine prints the input data, initial point, upper and lower bounds of each variable, machine precision, as well as the headings of the output.

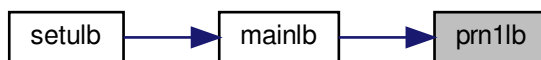
Parameters

<i>n</i>	On entry <i>n</i> is the number of variables. On exit <i>n</i> is unchanged.
<i>m</i>	On entry <i>m</i> is the maximum number of variable metric corrections allowed in the limited memory matrix. On exit <i>m</i> is unchanged.
<i>l</i>	On entry <i>l</i> is the lower bound of <i>x</i> . On exit <i>l</i> is unchanged.
<i>u</i>	On entry <i>u</i> is the upper bound of <i>x</i> . On exit <i>u</i> is unchanged.
<i>x</i>	On entry <i>x</i> is an approximation to the solution. On exit <i>x</i> is the current approximation.
<i>iprint</i>	It controls the frequency and type of output generated: • <i>iprint</i><0 no output is generated; • <i>iprint</i>=0 print only one line at the last iteration; • 0<<i>iprint</i><99 print also <i>f</i> and $\text{proj } g$ every <i>iprint</i> iterations; • <i>iprint</i>=99 print details of every iteration except <i>n</i>-vectors; • <i>iprint</i>=100 print also the changes of active set and final <i>x</i>; • <i>iprint</i>>100 print details of every iteration including <i>x</i> and <i>g</i>; When <i>iprint</i> > 0, the file <i>iterate.dat</i> will be created to summarize the iteration.
<i>itfile</i>	unit number of <i>iterate.dat</i> file
<i>epsmch</i>	machine precision epsilon

Definition at line 39 of file prn1lb.f.

Referenced by mainlb().

Here is the caller graph for this function:



3.16 src/prn2lb.f File Reference

Functions/Subroutines

- subroutine [prn2lb](#) (*n*, *x*, *f*, *g*, *iprint*, *itfile*, *iter*, *nfgv*, *nact*, *sbgnrm*, *nseg*, *word*, *iword*, *iback*, *stp*, *xstep*)

This subroutine prints out new information after a successful line search.

3.16.1 Function/Subroutine Documentation

3.16.1.1 prn2lb() `subroutine prn2lb (`
 `integer n,`
 `double precision, dimension(n) x,`
 `double precision f,`
 `double precision, dimension(n) g,`
 `integer iprint,`
 `integer itfile,`
 `integer iter,`
 `integer nfgv,`
 `integer nact,`
 `double precision sbgnrm,`
 `integer nseg,`
 `character*3 word,`
 `integer iword,`
 `integer iback,`
 `double precision stp,`
 `double precision xstep )`

This subroutine prints out new information after a successful line search.

Parameters

<i>n</i>	On entry <i>n</i> is the number of variables. On exit <i>n</i> is unchanged.
<i>x</i>	On entry <i>x</i> is an approximation to the solution. On exit <i>x</i> is the current approximation.
<i>f</i>	On first entry <i>f</i> is unspecified. On final exit <i>f</i> is the value of the function at <i>x</i> .

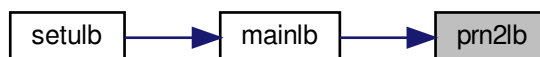
Parameters

<i>g</i>	On first entry <i>g</i> is unspecified. On final exit <i>g</i> is the value of the gradient at <i>x</i> .
<i>iprint</i>	It controls the frequency and type of output generated: • <i>iprint</i><0 no output is generated; • <i>iprint</i>=0 print only one line at the last iteration; • 0<<i>iprint</i><99 print also <i>f</i> and $\text{proj } g$ every <i>iprint</i> iterations; • <i>iprint</i>=99 print details of every iteration except <i>n</i>-vectors; • <i>iprint</i>=100 print also the changes of active set and final <i>x</i>; • <i>iprint</i>>100 print details of every iteration including <i>x</i> and <i>g</i>; When <i>iprint</i> > 0, the file <i>iterate.dat</i> will be created to summarize the iteration.
<i>itfile</i>	unit number of <i>iterate.dat</i> file
<i>iter</i>	Current outer iteration number; printed on the per-iterate summary line.
<i>nfgv</i>	Cumulative number of <i>f/g</i> evaluations across the whole run; logged into the <i>iterate.dat</i> record.
<i>nact</i>	Number of variables active at their bounds at the current iterate.
<i>sbgnrm</i>	Infinity norm of the projected gradient at <i>x</i> ; the natural first-order optimality measure printed alongside <i>f</i> .
<i>nseg</i>	Number of breakpoint segments traversed by cauchy at the current iteration; printed for diagnostic accounting.
<i>word</i>	On exit a 3-character status code summarising the subspace solution: 'con' (converged), 'bnd' (hit a bound), 'TNT' (truncated-Newton step used), or '—' otherwise. Determined from <i>iword</i> .
<i>iword</i>	Status code from <i>subsm</i> : 0 = subspace minimisation converged, 1 = stopped at a bound, 5 = truncated-Newton step used. Maps to <i>word</i> .
<i>iback</i>	Number of backtracks the line search performed.
<i>stp</i>	Final step length accepted by the line search.
<i>xstep</i>	$\text{stp} * d $ – the actual length of the step in <i>x</i> -space.

Definition at line 53 of file *prn2lb.f*.

Referenced by *mainlb()*.

Here is the caller graph for this function:



3.17 src/prn3lb.f File Reference

Functions/Subroutines

- subroutine [prn3lb](#) (*n*, *x*, *f*, *task*, *iprint*, *info*, *itfile*, *iter*, *nfgv*, *nintol*, *nskip*, *nact*, *sbgnrm*, *time*, *nseg*, *word*, *iback*, *stp*, *xstep*, *k*, *cachyt*, *sbttime*, *lnscht*)

This subroutine prints out information when either a built-in convergence test is satisfied or when an error message is generated.

3.17.1 Function/Subroutine Documentation

3.17.1.1 prn3lb() subroutine prn3lb (
integer *n*,
double precision, dimension(*n*) *x*,
double precision *f*,
character*60 *task*,
integer *iprint*,
integer *info*,
integer *itfile*,
integer *iter*,
integer *nfgv*,
integer *nintol*,
integer *nskip*,
integer *nact*,
double precision *sbgnrm*,
double precision *time*,
integer *nseg*,
character*3 *word*,
integer *iback*,
double precision *stp*,
double precision *xstep*,
integer *k*,
double precision *cachyt*,
double precision *sbttime*,
double precision *lnscht*)

This subroutine prints out information when either a built-in convergence test is satisfied or when an error message is generated.

Parameters

<i>n</i>	On entry <i>n</i> is the number of variables. On exit <i>n</i> is unchanged.
<i>x</i>	On entry <i>x</i> is an approximation to the solution. On exit <i>x</i> is the current approximation.
<i>f</i>	On first entry <i>f</i> is unspecified. On final exit <i>f</i> is the value of the function at <i>x</i> .
<i>task</i>	working string indicating the current job when entering and leaving this subroutine.
<i>iprint</i>	Console output flag; same convention as in setulb. Negative suppresses all output. ≥ 0 prints the per-run summary. ≥ 1 also writes to iterate.dat. ≥ 100 also prints the final <i>x</i> vector.
<i>info</i>	Termination status. 0 on normal convergence. Negative values map to specific error messages: -1, -2 (Cholesky not pos.def. in formk's 1st/2nd factor), -3 (formt's Cholesky), -4 (non-descent direction caught by Insr1b), -5 (>10 evals in a line search), -6 (invalid nbd(<i>k</i>)), -7 ($l(k) > u(k)$), -8 (singular triangular system), -9 (>20 evals in the last line search).
<i>itfile</i>	unit number of iterate.dat file
<i>iter</i>	Total number of outer iterations.
<i>nfgv</i>	Total number of f/g evaluations across the run.
<i>nintol</i>	Total number of breakpoint segments traversed by cauchy.

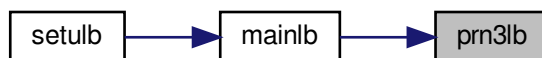
Parameters

<i>nskip</i>	Number of L-BFGS updates that were skipped because the curvature condition failed (s'y too small).
<i>nact</i>	Number of active bounds at the final generalised Cauchy point.
<i>sbgnrm</i>	Final infinity norm of the projected gradient at x.
<i>time</i>	Total wall-clock time of the optimisation run, in seconds.
<i>nseg</i>	Number of breakpoint segments traversed at the LAST cauchy call (used in the iterate.dat record on abnormal exits).
<i>word</i>	3-character status code from prn2lb: 'con', 'bnd', 'TNT', or '—' (see prn2lb.f). Logged in iterate.dat on info=-4 or info=-9 termination.
<i>iback</i>	Number of backtracks in the LAST line search.
<i>stp</i>	Final step length from the LAST line search.
<i>xstep</i>	$stp * d $ from the LAST line search (i.e. the actual step length in x-space).
<i>k</i>	Index of the offending parameter when info is -6 (invalid nbd) or -7 (infeasible bound). Set by errclb.
<i>cachyt</i>	Cumulative wall-clock time spent in cauchy, in seconds.
<i>sbtme</i>	Cumulative wall-clock time spent in subsm, in seconds.
<i>lnscht</i>	Cumulative wall-clock time spent in lnslb, in seconds.

Definition at line 56 of file prn3lb.f.

Referenced by mainlb().

Here is the caller graph for this function:



3.18 src/projgr.f File Reference

Functions/Subroutines

- subroutine [projgr](#) (n, l, u, nbd, x, g, sbgnrm)

This subroutine computes the infinity norm of the projected gradient.

3.18.1 Function/Subroutine Documentation

3.18.1.1 projgr() `subroutine projgr (`
 integer *n*,
 double precision, dimension(n) *l*,
 double precision, dimension(n) *u*,
 integer, dimension(n) *nbd*,
 double precision, dimension(n) *x*,
 double precision, dimension(n) *g*,
 double precision *sbgnrm*)

This subroutine computes the infinity norm of the projected gradient.

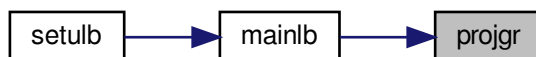
Parameters

<i>n</i>	On entry <i>n</i> is the number of variables. On exit <i>n</i> is unchanged.
<i>l</i>	On entry <i>l</i> is the lower bound of <i>x</i> . On exit <i>l</i> is unchanged.
<i>u</i>	On entry <i>u</i> is the upper bound of <i>x</i> . On exit <i>u</i> is unchanged.
<i>nbd</i>	On entry <i>nbd</i> represents the type of bounds imposed on the variables, and must be specified as follows: <i>nbd</i> (<i>i</i>)= • 0 if <i>x</i>(<i>i</i>) is unbounded, • 1 if <i>x</i>(<i>i</i>) has only a lower bound, • 2 if <i>x</i>(<i>i</i>) has both lower and upper bounds, • 3 if <i>x</i>(<i>i</i>) has only an upper bound. On exit <i>nbd</i> is unchanged.
<i>x</i>	On entry <i>x</i> is an approximation to the solution. On exit <i>x</i> is unchanged.
<i>g</i>	On entry <i>g</i> is the gradient. On exit <i>g</i> is unchanged.
<i>sbgnrm</i>	infinity norm of projected gradient

Definition at line 33 of file projgr.f.

Referenced by mainlb().

Here is the caller graph for this function:



3.19 src/setulb.f File Reference

Functions/Subroutines

- subroutine [setulb](#) (*n*, *m*, *x*, *l*, *u*, *nbd*, *f*, *g*, *factr*, *pgtol*, *wa*, *iwa*, *task*, *iprint*, *csave*, *lsave*, *isave*, *dsave*)
*This subroutine partitions the working arrays *wa* and *iwa*, and then uses the limited memory BFGS method to solve the bound constrained optimization problem by calling *mainlb*.*

3.19.1 Function/Subroutine Documentation

3.19.1.1 setulb() subroutine setulb (
integer *n*,
integer *m*,
double precision, dimension(*n*) *x*,
double precision, dimension(*n*) *l*,
double precision, dimension(*n*) *u*,
integer, dimension(*n*) *nbd*,
double precision *f*,
double precision, dimension(*n*) *g*,
double precision *factr*,
double precision *pgtol*,
double precision, dimension(2*m*n + 5*n + 11*m*m + 8*m) *wa*,
integer, dimension(3*n) *iwa*,
character*60 *task*,
integer *iprint*,
character*60 *csave*,
logical, dimension(4) *lsave*,
integer, dimension(44) *isave*,
double precision, dimension(29) *dsave*)

This subroutine partitions the working arrays *wa* and *iwa*, and then uses the limited memory BFGS method to solve the bound constrained optimization problem by calling *mainlb*. (The direct method will be used in the subspace minimization.)

Parameters

<i>n</i>	On entry <i>n</i> is the dimension of the problem. On exit <i>n</i> is unchanged.
<i>m</i>	On entry <i>m</i> is the maximum number of variable metric corrections used to define the limited memory matrix. On exit <i>m</i> is unchanged.
<i>x</i>	On entry <i>x</i> is an approximation to the solution. On exit <i>x</i> is the current approximation.
<i>l</i>	On entry <i>l</i> is the lower bound on <i>x</i> . On exit <i>l</i> is unchanged.
<i>u</i>	On entry <i>u</i> is the upper bound on <i>x</i> . On exit <i>u</i> is unchanged.
<i>nbd</i>	On entry <i>nbd</i> represents the type of bounds imposed on the variables, and must be specified as follows: <i>nbd</i> (<i>i</i>)= • 0 if <i>x</i>(<i>i</i>) is unbounded, • 1 if <i>x</i>(<i>i</i>) has only a lower bound, • 2 if <i>x</i>(<i>i</i>) has both lower and upper bounds, and • 3 if <i>x</i>(<i>i</i>) has only an upper bound. On exit <i>nbd</i> is unchanged.
<i>f</i>	On first entry <i>f</i> is unspecified. On final exit <i>f</i> is the value of the function at <i>x</i> .
<i>g</i>	On first entry <i>g</i> is unspecified. On final exit <i>g</i> is the value of the gradient at <i>x</i> .

Parameters

<i>factr</i>	On entry <i>factr</i> ≥ 0 is specified by the user. The iteration will stop when $(f^k - f^{k+1})/\max\{ f^k , f^{k+1} , 1\} \leq \text{factr} \cdot \text{epsmch}$ where <i>epsmch</i> is the machine precision, which is automatically generated by the code. Typical values for <i>factr</i>: • 1.d+12 for low accuracy; • 1.d+7 for moderate accuracy; • 1.d+1 for extremely high accuracy. On exit <i>factr</i> is unchanged.
<i>pgtol</i>	On entry <i>pgtol</i> ≥ 0 is specified by the user. The iteration will stop when $\max\{ \text{proj } g_i \mid i = 1, \dots, n\} \leq \text{pgtol}$ where <i>pg_i</i> is the <i>i</i>th component of the projected gradient. On exit <i>pgtol</i> is unchanged.
<i>wa</i>	working array
<i>iwa</i>	working array
<i>task</i>	working string indicating the current job when entering and quitting this subroutine.
<i>iprint</i>	Must be set by the user. It controls the frequency and type of output generated: • <i>iprint</i> < 0 no output is generated; • <i>iprint</i> = 0 print only one line at the last iteration; • 0 < <i>iprint</i> < 99 print also <i>f</i> and $\text{proj } g$ every <i>iprint</i> iterations; • <i>iprint</i> = 99 print details of every iteration except <i>n</i>-vectors; • <i>iprint</i> = 100 print also the changes of active set and final <i>x</i>; • <i>iprint</i> > 100 print details of every iteration including <i>x</i> and <i>g</i>; When <i>iprint</i> > 0, the file <i>iterate.dat</i> will be created to summarize the iteration.
<i>csave</i>	working string
<i>lsave</i>	working array; On exit with 'task' = NEW_X, the following information is available: • If <i>lsave</i>(1) = .true. then the initial <i>X</i> has been replaced by its projection in the feasible set; • If <i>lsave</i>(2) = .true. then the problem is constrained; • If <i>lsave</i>(3) = .true. then each variable has upper and lower bounds;

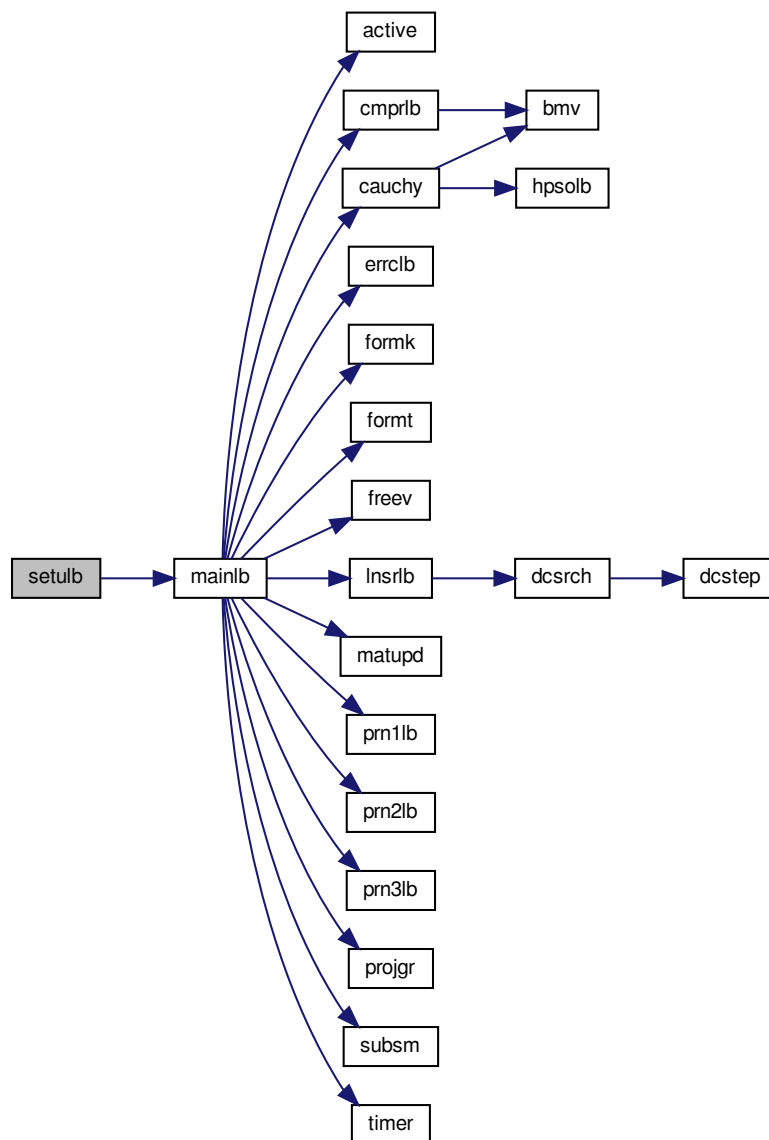
Parameters

<i>isave</i>	working array; On exit with 'task' = NEW_X, the following information is available: • isave(22) = the total number of intervals explored in the search of Cauchy points; • isave(26) = the total number of skipped BFGS updates before the current iteration; • isave(30) = the number of current iteration; • isave(31) = the total number of BFGS updates prior the current iteration; • isave(33) = the number of intervals explored in the search of Cauchy point in the current iteration; • isave(34) = the total number of function and gradient evaluations; • isave(36) = the number of function value or gradient evaluations in the current iteration; • if isave(37) = 0 then the subspace argmin is within the box; • if isave(37) = 1 then the subspace argmin is beyond the box; • isave(38) = the number of free variables in the current iteration; • isave(39) = the number of active constraints in the current iteration; • $n + 1 - \text{isave}(40)$ = the number of variables leaving the set of active constraints in the current iteration; • isave(41) = the number of variables entering the set of active constraints in the current iteration.
<i>dsave</i>	working array; On exit with 'task' = NEW_X, the following information is available: • dsave(1) = current 'theta' in the BFGS matrix; • dsave(2) = $f(x)$ in the previous iteration; • dsave(3) = $\text{factr} * \text{epsmch}$; • dsave(4) = 2-norm of the line search direction vector; • dsave(5) = the machine precision epsmch generated by the code; • dsave(7) = the accumulated time spent on searching for Cauchy points; • dsave(8) = the accumulated time spent on subspace minimization; • dsave(9) = the accumulated time spent on line search; • dsave(11) = the slope of the line search function at the current point of line search; • dsave(12) = the maximum relative step length imposed in line search; • dsave(13) = the infinity norm of the projected gradient; • dsave(14) = the relative step length in the line search; • dsave(15) = the slope of the line search function at the starting point of the line search; • dsave(16) = the square of the 2-norm of the line search direction vector.

Definition at line 188 of file setulb.f.

References mainlb().

Here is the call graph for this function:



3.20 src/subsm.f File Reference

Functions/Subroutines

- subroutine [subsm](#) (n, m, nsub, ind, l, u, nbd, x, d, xp, ws, wy, theta, xx, gg, col, head, iword, wv, wn, iprint, info)
Performs the subspace minimization.

3.20.1 Function/Subroutine Documentation

3.20.1.1 subsm() subroutine subsm (
integer *n*,
integer *m*,
integer *nsub*,
integer, dimension(*nsub*) *ind*,
double precision, dimension(*n*) *l*,
double precision, dimension(*n*) *u*,
integer, dimension(*n*) *nbd*,
double precision, dimension(*n*) *x*,
double precision, dimension(*n*) *d*,
double precision, dimension(*n*) *xp*,
double precision, dimension(*n*, *m*) *ws*,
double precision, dimension(*n*, *m*) *wy*,
double precision *theta*,
double precision, dimension(*n*) *xx*,
double precision, dimension(*n*) *gg*,
integer *col*,
integer *head*,
integer *iword*,
double precision, dimension(2**m*) *wv*,
double precision, dimension(2**m*, 2**m*) *wn*,
integer *iprint*,
integer *info*)

Given *xcp*, *l*, *u*, *r*, an index set that specifies the active set at *xcp*, and an L-BFGS matrix *B* (in terms of *WY*, *WS*, *SY*, *WT*, *head*, *col*, and *theta*), this subroutine computes an approximate solution of the subspace problem

$$(P) \min Q(x) = r'(x-xcp) + 1/2 (x-xcp)' B (x-xcp)$$

subject to $l \leq x \leq u$ $x_i = xcp_i$ for all *i* in *A(xcp)*

along the subspace unconstrained Newton direction

$$d = -(Z'BZ)^{-1} r.$$

The formula for the Newton direction, given the L-BFGS matrix and the Sherman-Morrison formula, is

$$d = (1/\theta)r + (1/\theta^2) Z'WK^{-1}W'Z r.$$

where $K = [-D -Y'ZZ'Y/\theta L_a -R_z'] [L_a -R_z \theta S'AA'S]$

Note that this procedure for computing *d* differs from that described in [1]. One can show that the matrix *K* is equal to the matrix $M^+[-1]N$ in that paper.

Parameters

<i>n</i>	On entry <i>n</i> is the dimension of the problem. On exit <i>n</i> is unchanged.
<i>m</i>	On entry <i>m</i> is the maximum number of variable metric corrections used to define the limited memory matrix. On exit <i>m</i> is unchanged.
<i>nsub</i>	On entry <i>nsub</i> is the number of free variables. On exit <i>nsub</i> is unchanged.
<i>ind</i>	On entry <i>ind</i> specifies the coordinate indices of free variables. On exit <i>ind</i> is unchanged.
<i>l</i>	On entry <i>l</i> is the lower bound of <i>x</i> . On exit <i>l</i> is unchanged.

Parameters

<i>u</i>	On entry <i>u</i> is the upper bound of <i>x</i> . On exit <i>u</i> is unchanged.
<i>nbd</i>	On entry <i>nbd</i> represents the type of bounds imposed on the variables, and must be specified as follows: $nbd(i) =$ • 0 if <i>x</i>(<i>i</i>) is unbounded, • 1 if <i>x</i>(<i>i</i>) has only a lower bound, • 2 if <i>x</i>(<i>i</i>) has both lower and upper bounds, and • 3 if <i>x</i>(<i>i</i>) has only an upper bound. On exit <i>nbd</i> is unchanged.
<i>x</i>	On entry <i>x</i> specifies the Cauchy point <i>xcp</i> . On exit <i>x</i> (<i>i</i>) is the minimizer of <i>Q</i> over the subspace of free variables.
<i>d</i>	On entry <i>d</i> is the reduced gradient of <i>Q</i> at <i>xcp</i> . On exit <i>d</i> is the Newton direction of <i>Q</i> .
<i>xp</i>	used to safeguard the projected Newton direction
<i>xx</i>	On entry it holds the current iterate. On output it is unchanged.
<i>gg</i>	On entry it holds the gradient at the current iterate. On output it is unchanged.
<i>ws</i>	On entry this stores <i>S</i> , a set of <i>s</i> -vectors, that defines the limited memory BFGS matrix. On exit this array is unchanged.
<i>wy</i>	On entry this stores <i>Y</i> , a set of <i>y</i> -vectors, that defines the limited memory BFGS matrix. On exit this array is unchanged.
<i>theta</i>	On entry <i>theta</i> is the scaling factor specifying $B_0 = \text{theta } I$. On exit <i>theta</i> is unchanged.
<i>col</i>	On entry <i>col</i> is the actual number of variable metric corrections stored so far. On exit <i>col</i> is unchanged.
<i>head</i>	On entry <i>head</i> is the location of the first <i>s</i> -vector (or <i>y</i> -vector) in <i>S</i> (or <i>Y</i>). On exit <i>col</i> is unchanged.
<i>word</i>	On entry <i>word</i> is unspecified. On exit <i>word</i> specifies the status of the subspace solution. <i>word</i> = • 0 if the solution is in the box, • 1 if some bound is encountered.
<i>wv</i>	working array
<i>wn</i>	On entry the upper triangle of <i>wn</i> stores the LEL^T factorization of the indefinite matrix $K = [-D -Y'ZZ'Y/\text{theta } L_a -R_z'] [L_a -R_z \text{theta} *S'AA'S]$ where $E = [-I \ 0] [\ 0 \ I]$ On exit <i>wn</i> is unchanged.

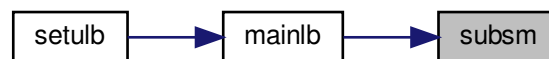
Parameters

<i>iprint</i>	must be set by the user; It controls the frequency and type of output generated: • <code>iprint<0</code> no output is generated; • <code>iprint=0</code> print only one line at the last iteration; • <code>0<iprint<99</code> print also <code>f</code> and <code> proj g </code> every <code>iprint</code> iterations; • <code>iprint=99</code> print details of every iteration except <code>n</code>-vectors; • <code>iprint=100</code> print also the changes of active set and final <code>x</code>; • <code>iprint>100</code> print details of every iteration including <code>x</code> and <code>g</code>; When <code>iprint > 0</code>, the file <code>iterate.dat</code> will be created to summarize the iteration.
<i>info</i>	On entry <code>info</code> is unspecified. On exit <code>info</code> = • 0 for normal return, • nonzero for abnormal return when the matrix <code>K</code> is ill-conditioned.

Definition at line 124 of file `subsm.f`.

Referenced by `mainlb()`.

Here is the caller graph for this function:



3.21 src/timer.f File Reference

Functions/Subroutines

- subroutine `timer` (`ttime`)
This routine computes cpu time in double precision.

3.21.1 Function/Subroutine Documentation

3.21.1.1 timer() `subroutine timer (`
`double precision ttime )`

This routine computes cpu time in double precision; it makes use of the intrinsic `f90 cpu_time` therefore a conversion type is needed.

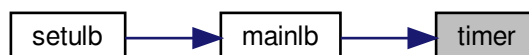
Parameters

<i>time</i>	CPU time in double precision
-------------	------------------------------

Definition at line 10 of file timer.f.

Referenced by mainlb().

Here is the caller graph for this function:



4 Example Documentation

4.1 driver1.f

This simple driver demonstrates how to call the L-BFGS-B code to solve a sample problem (the extended Rosenbrock function subject to bounds on the variables). The dimension n of this problem is variable. (Fortran-77 version)

```

1 c> \file driver1.f
2
3 c
4 c L-BFGS-B is released under the "New BSD License" (aka "Modified BSD License"
5 c or "3-clause license")
6 c Please read attached file License.txt
7 c
8 c DRIVER 1 in Fortran 77
9 c
10 c -----
11 c SIMPLE DRIVER FOR L-BFGS-B (version 3.0)
12 c -----
13 c L-BFGS-B is a code for solving large nonlinear optimization
14 c problems with simple bounds on the variables.
15 c
16 c The code can also be used for unconstrained problems and is
17 c as efficient for these problems as the earlier limited memory
18 c code L-BFGS.
19 c
20 c This is the simplest driver in the package. It uses all the
21 c default settings of the code.
22 c
23 c
24 c References:
25 c
26 c [1] R. H. Byrd, P. Lu, J. Nocedal and C. Zhu, "A limited
27 c memory algorithm for bound constrained optimization",
28 c SIAM J. Scientific Computing 16 (1995), no. 5, pp. 1190--1208.
29 c
30 c [2] C. Zhu, R.H. Byrd, P. Lu, J. Nocedal, "L-BFGS-B: FORTRAN
31 c Subroutines for Large Scale Bound Constrained Optimization"
32 c Tech. Report, NAM-11, EECS Department, Northwestern University,
33 c 1994.
34 c
35 c
36 c (Postscript files of these papers are available via anonymous
37 c ftp to eecs.nwu.edu in the directory pub/lbfgs/lbfgs_bcm.)
38 c
39 c * * *
40 c
41 c March 2011 (latest revision)

```

```

42 c      Optimization Center at Northwestern University
43 c      Instituto Tecnológico Autónomo de México
44 c
45 c      Jorge Nocedal and Jose Luis Morales, Remark on "Algorithm 778:
46 c      L-BFGS-B: Fortran Subroutines for Large-Scale Bound Constrained
47 c      Optimization" (2011). To appear in ACM Transactions on
48 c      Mathematical Software,
49
50 c      -----
51 c      DESCRIPTION OF THE VARIABLES IN L-BFGS-B
52 c      -----
53 c
54 c      n is an INTEGER variable that must be set by the user to the
55 c      number of variables. It is not altered by the routine.
56 c
57 c      m is an INTEGER variable that must be set by the user to the
58 c      number of corrections used in the limited memory matrix.
59 c      It is not altered by the routine. Values of m < 3 are
60 c      not recommended, and large values of m can result in excessive
61 c      computing time. The range 3 <= m <= 20 is recommended.
62 c
63 c      x is a DOUBLE PRECISION array of length n. On initial entry
64 c      it must be set by the user to the values of the initial
65 c      estimate of the solution vector. Upon successful exit, it
66 c      contains the values of the variables at the best point
67 c      found (usually an approximate solution).
68 c
69 c      l is a DOUBLE PRECISION array of length n that must be set by
70 c      the user to the values of the lower bounds on the variables. If
71 c      the i-th variable has no lower bound, l(i) need not be defined.
72 c
73 c      u is a DOUBLE PRECISION array of length n that must be set by
74 c      the user to the values of the upper bounds on the variables. If
75 c      the i-th variable has no upper bound, u(i) need not be defined.
76 c
77 c      nbd is an INTEGER array of dimension n that must be set by the
78 c      user to the type of bounds imposed on the variables:
79 c      nbd(i)=0 if x(i) is unbounded,
80 c      1 if x(i) has only a lower bound,
81 c      2 if x(i) has both lower and upper bounds,
82 c      3 if x(i) has only an upper bound.
83 c
84 c      f is a DOUBLE PRECISION variable. If the routine setulb returns
85 c      with task(1:2)= 'FG', then f must be set by the user to
86 c      contain the value of the function at the point x.
87 c
88 c      g is a DOUBLE PRECISION array of length n. If the routine setulb
89 c      returns with task(1:2)= 'FG', then g must be set by the user to
90 c      contain the components of the gradient at the point x.
91 c
92 c      factr is a DOUBLE PRECISION variable that must be set by the user.
93 c      It is a tolerance in the termination test for the algorithm.
94 c      The iteration will stop when
95 c
96 c      (f^k - f^{k+1})/max{|f^k|,|f^{k+1}|,1} <= factr*epsmach
97 c
98 c      where epsmach is the machine precision which is automatically
99 c      generated by the code. Typical values for factr on a computer
100 c      with 15 digits of accuracy in double precision are:
101 c      factr=1.d+12 for low accuracy;
102 c      1.d+7 for moderate accuracy;
103 c      1.d+1 for extremely high accuracy.
104 c      The user can suppress this termination test by setting factr=0.
105 c
106 c      pgtol is a double precision variable.
107 c      On entry pgtol >= 0 is specified by the user. The iteration
108 c      will stop when
109 c
110 c      max{|proj g_i | i = 1, ..., n} <= pgtol
111 c
112 c      where pg_i is the ith component of the projected gradient.
113 c      The user can suppress this termination test by setting pgtol=0.
114 c
115 c      wa is a DOUBLE PRECISION array of length
116 c      (2nmax + 5)nmax + 11mmax^2 + 8mmmax used as workspace.
117 c      This array must not be altered by the user.
118 c
119 c      iwa is an INTEGER array of length 3nmax used as
120 c      workspace. This array must not be altered by the user.
121 c
122 c      task is a CHARACTER string of length 60.
123 c      On first entry, it must be set to 'START'.
124 c      On a return with task(1:2)= 'FG', the user must evaluate the
125 c      function f and gradient g at the returned value of x.
126 c      On a return with task(1:5)= 'NEW_X', an iteration of the
127 c      algorithm has concluded, and f and g contain f(x) and g(x)
128 c      respectively. The user can decide whether to continue or stop

```

```

129 c         the iteration.
130 c     When
131 c         task(1:4)='CONV', the termination test in L-BFGS-B has been
132 c         satisfied;
133 c         task(1:4)='ABNO', the routine has terminated abnormally
134 c         without being able to satisfy the termination conditions,
135 c         x contains the best approximation found,
136 c         f and g contain f(x) and g(x) respectively;
137 c         task(1:5)='ERROR', the routine has detected an error in the
138 c         input parameters;
139 c     On exit with task = 'CONV', 'ABNO' or 'ERROR', the variable task
140 c     contains additional information that the user can print.
141 c     This array should not be altered unless the user wants to
142 c     stop the run for some reason. See driver2 or driver3
143 c     for a detailed explanation on how to stop the run
144 c     by assigning task(1:4)='STOP' in the driver.
145 c
146 c     iprint is an INTEGER variable that must be set by the user.
147 c     It controls the frequency and type of output generated:
148 c         iprint<0    no output is generated;
149 c         iprint=0     print only one line at the last iteration;
150 c         0<iprint<99 print also f and |proj g| every iprint iterations;
151 c         iprint=99    print details of every iteration except n-vectors;
152 c         iprint=100   print also the changes of active set and final x;
153 c         iprint>100   print details of every iteration including x and g;
154 c     When iprint > 0, the file iterate.dat will be created to
155 c     summarize the iteration.
156 c
157 c     csave is a CHARACTER working array of length 60.
158 c
159 c     lsave is a LOGICAL working array of dimension 4.
160 c     On exit with task = 'NEW_X', the following information is
161 c     available:
162 c         lsave(1) = .true. the initial x did not satisfy the bounds;
163 c         lsave(2) = .true. the problem contains bounds;
164 c         lsave(3) = .true. each variable has upper and lower bounds.
165 c
166 c     isave is an INTEGER working array of dimension 44.
167 c     On exit with task = 'NEW_X', it contains information that
168 c     the user may want to access:
169 c         isave(30) = the current iteration number;
170 c         isave(34) = the total number of function and gradient
171 c         evaluations;
172 c         isave(36) = the number of function value or gradient
173 c         evaluations in the current iteration;
174 c         isave(38) = the number of free variables in the current
175 c         iteration;
176 c         isave(39) = the number of active constraints at the current
177 c         iteration;
178 c
179 c     see the subroutine setulb.f for a description of other
180 c     information contained in isave
181 c
182 c     dsave is a DOUBLE PRECISION working array of dimension 29.
183 c     On exit with task = 'NEW_X', it contains information that
184 c     the user may want to access:
185 c         dsave(2) = the value of f at the previous iteration;
186 c         dsave(5) = the machine precision epsmch generated by the code;
187 c         dsave(13) = the infinity norm of the projected gradient;
188 c
189 c     see the subroutine setulb.f for a description of other
190 c     information contained in dsave
191 c
192 c     -----
193 c     END OF THE DESCRIPTION OF THE VARIABLES IN L-BFGS-B
194 c     -----
195
196 program driver
197 use json_writer
198
199 c     This simple driver demonstrates how to call the L-BFGS-B code to
200 c     solve a sample problem (the extended Rosenbrock function
201 c     subject to bounds on the variables). The dimension n of this
202 c     problem is variable.
203
204 integer          nmax, mmax
205 parameter(nmax=1024, mmax=17)
206 c     nmax is the dimension of the largest problem to be solved.
207 c     mmax is the maximum number of limited memory corrections.
208
209 c     Declare the variables needed by the code.
210 c     A description of all these variables is given at the end of
211 c     the driver.
212
213 character*60     task, csave
214 logical          lsave(4)
215 integer          n, m, iprint,

```

```

216      +          nbd(nmax), iwa(3*nmax), isave(44)
217      double precision f, factr, pgtol,
218      +          x(nmax), l(nmax), u(nmax), g(nmax), dsave(29),
219      +          wa(2*mmax*nmax + 5*nmax + 11*mmax*mmax + 8*mmax)
220
221 c      Declare a few additional variables for this sample problem.
222
223      double precision t1, t2
224      integer          i
225
226 c      Test instrumentation: dump per-iteration state to JSON when the
227 c      environment variable LBFGSB_JSON_OUTPUT is set. No-op otherwise.
228      character*512     lbfgsb_json
229      logical           json_active
230
231 c      We wish to have output at every iteration.
232
233      iprint = 1
234
235 c      We specify the tolerances in the stopping criteria.
236
237      factr=1.0d+7
238      pgtol=1.0d-5
239
240 c      We specify the dimension n of the sample problem and the number
241 c      m of limited memory corrections stored. (n and m should not
242 c      exceed the limits nmax and mmax respectively.)
243
244      n=25
245      m=5
246
247 c      We now provide nbd which defines the bounds on the variables:
248 c      l specifies the lower bounds,
249 c      u specifies the upper bounds.
250
251 c      First set bounds on the odd-numbered variables.
252
253      do 10 i=1,n,2
254          nbd(i)=2
255          l(i)=1.0d0
256          u(i)=1.0d2
257 10  continue
258
259 c      Next set bounds on the even-numbered variables.
260
261      do 12 i=2,n,2
262          nbd(i)=2
263          l(i)=-1.0d2
264          u(i)=1.0d2
265 12  continue
266
267 c      We now define the starting point.
268
269      do 14 i=1,n
270          x(i)=3.0d0
271 14  continue
272
273      write (6,16)
274 16  format(/,5x, 'Solving sample problem.',
275      +      /,5x, ' (f = 0.0 at the optimal solution.)',/)
276
277 c      We start the iteration by initializing task.
278 c
279      task = 'START'
280
281 c      Test instrumentation: enabled when LBFGSB_JSON_OUTPUT is set.
282      call get_environment_variable('LBFGSB_JSON_OUTPUT', lbfgsb_json)
283      json_active = (len_trim(lbfgsb_json) .gt. 0)
284
285 c      ----- the beginning of the loop -----
286
287 111 continue
288
289 c      This is the call to the L-BFGS-B code.
290
291      call setulb(n,m,x,l,u,nbd,f,g,factr,pgtol,wa,iwa,task,iprint,
292      +          csave,lsave,isave,dsave)
293
294      if (task(1:2) .eq. 'FG') then
295 c      the minimization routine has returned to request the
296 c      function f and gradient g values at the current x.
297
298 c      Compute function value f for the sample problem.
299
300      f=.25d0*(x(1)-1.d0)**2
301      do 20 i=2,n
302          f=f+(x(i)-x(i-1)**2)**2

```

```

303 20      continue
304         f=4.d0*f
305
306 c        Compute gradient g for the sample problem.
307
308         t1=x(2)-x(1)**2
309         g(1)=2.d0*(x(1)-1.d0)-1.6d1*x(1)*t1
310         do 22 i=2,n-1
311             t2=t1
312             t1=x(i+1)-x(i)**2
313             g(i)=8.d0*t2-1.6d1*x(i)*t1
314 22      continue
315         g(n)=8.d0*t1
316
317 c        go back to the minimization routine.
318         goto 111
319     endif
320 c
321     if (task(1:5) .eq. 'NEW_X') goto 111
322 c        the minimization routine has returned with a new iterate,
323 c        and we have opted to continue the iteration.
324
325 c        ----- the end of the loop -----
326
327 c        If task is neither FG nor NEW_X we terminate execution.
328
329         if (json_active) then
330             call json_write_aggregate(trim(lbfgsb_json), task, f,
331 +                                     dsave(13), n, x)
332         endif
333
334         stop
335
336     end
337
338 c===== The end of driver1 =====

```

4.2 driver1.f90

This simple driver demonstrates how to call the L-BFGS-B code to solve a sample problem (the extended Rosenbrock function subject to bounds on the variables). The dimension n of this problem is variable. (Fortran-90 version)

```

1 !> \file driver1.f90
2
3 !
4 ! L-BFGS-B is released under the "New BSD License" (aka "Modified BSD License"
5 ! or "3-clause license")
6 ! Please read attached file License.txt
7 !
8 !
9 !
10 ! ----- DRIVER1 in Fortran 90 -----
11 !
12 ! L-BFGS-B is a code for solving large nonlinear optimization
13 ! problems with simple bounds on the variables.
14 !
15 ! The code can also be used for unconstrained problems and is
16 ! as efficient for these problems as the earlier limited memory
17 ! code L-BFGS.
18 !
19 ! This is the simplest driver in the package. It uses all the
20 ! default settings of the code.
21 !
22 !
23 ! References:
24 !
25 ! [1] R. H. Byrd, P. Lu, J. Nocedal and C. Zhu, "A limited
26 ! memory algorithm for bound constrained optimization",
27 ! SIAM J. Scientific Computing 16 (1995), no. 5, pp. 1190--1208.
28 !
29 ! [2] C. Zhu, R.H. Byrd, P. Lu, J. Nocedal, "L-BFGS-B: FORTRAN
30 ! Subroutines for Large Scale Bound Constrained Optimization"
31 ! Tech. Report, NAM-11, EECS Department, Northwestern University,
32 ! 1994.
33 !
34 !
35 ! (Postscript files of these papers are available via anonymous
36 ! ftp to eeecs.nwu.edu in the directory pub/lbfgs/lbfgs_bcm.)
37 !
38 ! * * *
39 !
40 ! March 2011 (latest revision)
41 ! Optimization Center at Northwestern University

```

```

42 !      Instituto Tecnologico Autonomo de Mexico
43 !
44 !      Jorge Nocedal and Jose Luis Morales
45 !
46 !      -----
47 !      DESCRIPTION OF THE VARIABLES IN L-BFGS-B
48 !      -----
49 !
50 !      n is an INTEGER variable that must be set by the user to the
51 !      number of variables.  It is not altered by the routine.
52 !
53 !      m is an INTEGER variable that must be set by the user to the
54 !      number of corrections used in the limited memory matrix.
55 !      It is not altered by the routine.  Values of m < 3 are
56 !      not recommended, and large values of m can result in excessive
57 !      computing time. The range 3 <= m <= 20 is recommended.
58 !
59 !      x is a DOUBLE PRECISION array of length n.  On initial entry
60 !      it must be set by the user to the values of the initial
61 !      estimate of the solution vector.  Upon successful exit, it
62 !      contains the values of the variables at the best point
63 !      found (usually an approximate solution).
64 !
65 !      l is a DOUBLE PRECISION array of length n that must be set by
66 !      the user to the values of the lower bounds on the variables. If
67 !      the i-th variable has no lower bound, l(i) need not be defined.
68 !
69 !      u is a DOUBLE PRECISION array of length n that must be set by
70 !      the user to the values of the upper bounds on the variables. If
71 !      the i-th variable has no upper bound, u(i) need not be defined.
72 !
73 !      nbd is an INTEGER array of dimension n that must be set by the
74 !      user to the type of bounds imposed on the variables:
75 !      nbd(i)=0 if x(i) is unbounded,
76 !              1 if x(i) has only a lower bound,
77 !              2 if x(i) has both lower and upper bounds,
78 !              3 if x(i) has only an upper bound.
79 !
80 !      f is a DOUBLE PRECISION variable.  If the routine setulb returns
81 !      with task(1:2)= 'FG', then f must be set by the user to
82 !      contain the value of the function at the point x.
83 !
84 !      g is a DOUBLE PRECISION array of length n.  If the routine setulb
85 !      returns with taskb(1:2)= 'FG', then g must be set by the user to
86 !      contain the components of the gradient at the point x.
87 !
88 !      factr is a DOUBLE PRECISION variable that must be set by the user.
89 !      It is a tolerance in the termination test for the algorithm.
90 !      The iteration will stop when
91 !
92 !      (f^k - f^{k+1})/max{|f^k|,|f^{k+1}|,1} <= factr*epsmch
93 !
94 !      where epsmch is the machine precision which is automatically
95 !      generated by the code. Typical values for factr on a computer
96 !      with 15 digits of accuracy in double precision are:
97 !      factr=1.d+12 for low accuracy;
98 !              1.d+7  for moderate accuracy;
99 !              1.d+1  for extremely high accuracy.
100 !      The user can suppress this termination test by setting factr=0.
101 !
102 !      pgtol is a double precision variable.
103 !      On entry pgtol >= 0 is specified by the user.  The iteration
104 !      will stop when
105 !
106 !      max{|proj g_i | i = 1, ..., n} <= pgtol
107 !
108 !      where pg_i is the ith component of the projected gradient.
109 !      The user can suppress this termination test by setting pgtol=0.
110 !
111 !      wa is a DOUBLE PRECISION array of length
112 !      (2mmax + 5)nmax + 11mmax^2 + 8mmax used as workspace.
113 !      This array must not be altered by the user.
114 !
115 !      iwa is an INTEGER array of length 3nmax used as
116 !      workspace. This array must not be altered by the user.
117 !
118 !      task is a CHARACTER string of length 60.
119 !      On first entry, it must be set to 'START'.
120 !      On a return with task(1:2)='FG', the user must evaluate the
121 !      function f and gradient g at the returned value of x.
122 !      On a return with task(1:5)='NEW_X', an iteration of the
123 !      algorithm has concluded, and f and g contain f(x) and g(x)
124 !      respectively. The user can decide whether to continue or stop
125 !      the iteration.
126 !      When
127 !      task(1:4)='CONV', the termination test in L-BFGS-B has been
128 !      satisfied;

```

```

129 !      task(1:4)='ABNO', the routine has terminated abnormally
130 !          without being able to satisfy the termination conditions,
131 !          x contains the best approximation found,
132 !          f and g contain f(x) and g(x) respectively;
133 !      task(1:5)='ERROR', the routine has detected an error in the
134 !          input parameters;
135 !      On exit with task = 'CONV', 'ABNO' or 'ERROR', the variable task
136 !          contains additional information that the user can print.
137 !      This array should not be altered unless the user wants to
138 !          stop the run for some reason. See driver2 or driver3
139 !          for a detailed explanation on how to stop the run
140 !          by assigning task(1:4)='STOP' in the driver.
141 !
142 !      iprint is an INTEGER variable that must be set by the user.
143 !      It controls the frequency and type of output generated:
144 !          iprint<0    no output is generated;
145 !          iprint=0     print only one line at the last iteration;
146 !          0<iprint<99 print also f and |proj g| every iprint iterations;
147 !          iprint=99    print details of every iteration except n-vectors;
148 !          iprint=100   print also the changes of active set and final x;
149 !          iprint>100   print details of every iteration including x and g;
150 !      When iprint > 0, the file iterate.dat will be created to
151 !          summarize the iteration.
152 !
153 !      csave is a CHARACTER working array of length 60.
154 !
155 !      lsave is a LOGICAL working array of dimension 4.
156 !      On exit with task = 'NEW_X', the following information is
157 !          available:
158 !          lsave(1) = .true.  the initial x did not satisfy the bounds;
159 !          lsave(2) = .true.  the problem contains bounds;
160 !          lsave(3) = .true.  each variable has upper and lower bounds.
161 !
162 !      isave is an INTEGER working array of dimension 44.
163 !      On exit with task = 'NEW_X', it contains information that
164 !          the user may want to access:
165 !          isave(30) = the current iteration number;
166 !          isave(34) = the total number of function and gradient
167 !                      evaluations;
168 !          isave(36) = the number of function value or gradient
169 !                      evaluations in the current iteration;
170 !          isave(38) = the number of free variables in the current
171 !                      iteration;
172 !          isave(39) = the number of active constraints at the current
173 !                      iteration;
174 !
175 !          see the subroutine setulb.f for a description of other
176 !          information contained in isave
177 !
178 !      dsave is a DOUBLE PRECISION working array of dimension 29.
179 !      On exit with task = 'NEW_X', it contains information that
180 !          the user may want to access:
181 !          dsave(2)  = the value of f at the previous iteration;
182 !          dsave(5)  = the machine precision epsmch generated by the code;
183 !          dsave(13) = the infinity norm of the projected gradient;
184 !
185 !          see the subroutine setulb.f for a description of other
186 !          information contained in dsave
187 !
188 !      -----
189 !      END OF THE DESCRIPTION OF THE VARIABLES IN L-BFGS-B
190 !      -----
191 !
192 !      program driver
193 !      use json_writer
194 !
195 !      This simple driver demonstrates how to call the L-BFGS-B code to
196 !          solve a sample problem (the extended Rosenbrock function
197 !          subject to bounds on the variables). The dimension n of this
198 !          problem is variable.
199 !
200 !      implicit none
201 !
202 !      Declare variables and parameters needed by the code.
203 !      Note thar we wish to have output at every iteration.
204 !          iprint=1
205 !
206 !      We also specify the tolerances in the stopping criteria.
207 !          factr = 1.0d+7, pgtol = 1.0d-5
208 !
209 !      A description of all these variables is given at the beginning
210 !          of the driver
211 !
212 !      integer, parameter :: n = 25, m = 5, iprint = 1
213 !      integer, parameter :: dp = kind(1.0d0)
214 !      real(dp), parameter :: factr = 1.0d+7, pgtol = 1.0d-5
215 !

```

```

216     character(len=60)      :: task, csave
217     logical                :: lsave(4)
218     integer                :: isave(44)
219     real(dp)               :: f
220     real(dp)               :: dsave(29)
221     integer, allocatable   :: nbd(:), iwa(:)
222     real(dp), allocatable  :: x(:), l(:), u(:), g(:), wa(:)
223
224 !     Declare a few additional variables for this sample problem
225
226     real(dp)               :: t1, t2
227     integer                :: i
228
229 !     Test instrumentation: dump per-iteration state to JSON when the
230 !     environment variable LBFGSB_JSON_OUTPUT is set. No-op otherwise.
231     character(len=512)     :: lbfgsb_json
232     logical                :: json_active
233
234 !     Allocate dynamic arrays
235
236     allocate ( nbd(n), x(n), l(n), u(n), g(n) )
237     allocate ( iwa(3*n) )
238     allocate ( wa(2*m*n + 11*m*m + 5*n + 8*m) )
239 !
240     do 10 i=1, n, 2
241         nbd(i) = 2
242         l(i)   = 1.0d0
243         u(i)   = 1.0d2
244 10    continue
245
246 !     Next set bounds on the even-numbered variables.
247
248     do 12 i=2, n, 2
249         nbd(i) = 2
250         l(i)   = -1.0d2
251         u(i)   = 1.0d2
252 12    continue
253
254 !     We now define the starting point.
255
256     do 14 i=1, n
257         x(i) = 3.0d0
258 14    continue
259
260     write (6,16)
261 16    format(/,5x, 'Solving sample problem.', &
262            /,5x, ' (f = 0.0 at the optimal solution.)',/)
263
264 !     We start the iteration by initializing task.
265
266     task = 'START'
267
268 !     Test instrumentation: enabled when LBFGSB_JSON_OUTPUT is set.
269     call get_environment_variable('LBFGSB_JSON_OUTPUT', lbfgsb_json)
270     json_active = (len_trim(lbfgsb_json) > 0)
271
272 !     The beginning of the loop
273
274     do while(task(1:2).eq.'FG'.or.task.eq.'NEW_X'.or. &
275            task.eq.'START')
276
277 !     This is the call to the L-BFGS-B code.
278
279         call setulb ( n, m, x, l, u, nbd, f, g, factr, pgtol, &
280                    wa, iwa, task, iprint,&
281                    csave, lsave, isave, dsave )
282
283         if (task(1:2) .eq. 'FG') then
284
285             f=.25d0*( x(1)-1.d0 )**2
286             do 20 i=2, n
287                 f = f + ( x(i)-x(i-1) )**2 )**2
288 20            continue
289             f = 4.d0*f
290
291 !     Compute gradient g for the sample problem.
292
293             t1 = x(2) - x(1)**2
294             g(1) = 2.d0*(x(1) - 1.d0) - 1.6d1*x(1)*t1
295             do 22 i=2, n-1
296                 t2 = t1
297                 t1 = x(i+1) - x(i)**2
298                 g(i) = 8.d0*t2 - 1.6d1*x(i)*t1
299 22            continue
300             g(n) = 8.d0*t1
301
302         end if

```

```

303         end do
304
305 !       end of loop do while
306
307         if (json_active) &
308             call json_write_aggregate(trim(lbfgsb_json), task, f, dsave(13), n, x)
309
310     end program driver
311

```

4.3 driver2.f

This driver shows how to replace the default stopping test by other termination criteria. It also illustrates how to print the values of several parameters during the course of the iteration. The sample problem used here is the same as in DRIVER1 (the extended Rosenbrock function with bounds on the variables). (Fortran-77 version)

```

1 c> \file driver2.f
2
3 c
4 c L-BFGS-B is released under the "New BSD License" (aka "Modified BSD License"
5 c or "3-clause license")
6 c Please read attached file License.txt
7 c
8 c                                     DRIVER 2 in Fortran 77
9 c -----
10 c          CUSTOMIZED DRIVER FOR L-BFGS-B (version 3.0)
11 c -----
12 c
13 c     L-BFGS-B is a code for solving large nonlinear optimization
14 c     problems with simple bounds on the variables.
15 c
16 c     The code can also be used for unconstrained problems and is
17 c     as efficient for these problems as the earlier limited memory
18 c     code L-BFGS.
19 c
20 c     This driver illustrates how to control the termination of the
21 c     run and how to design customized output.
22 c
23 c References:
24 c
25 c     [1] R. H. Byrd, P. Lu, J. Nocedal and C. Zhu, "A limited
26 c     memory algorithm for bound constrained optimization",
27 c     SIAM J. Scientific Computing 16 (1995), no. 5, pp. 1190--1208.
28 c
29 c     [2] C. Zhu, R.H. Byrd, P. Lu, J. Nocedal, "L-BFGS-B: FORTRAN
30 c     Subroutines for Large Scale Bound Constrained Optimization"
31 c     Tech. Report, NAM-11, EECS Department, Northwestern University,
32 c     1994.
33 c
34 c
35 c     (Postscript files of these papers are available via anonymous
36 c     ftp to eecs.nwu.edu in the directory pub/lbfgs/lbfgs_bcm.)
37 c
38 c             * * *
39 c
40 c     February 2011 (latest revision)
41 c     Optimization Center at Northwestern University
42 c     Instituto Tecnológico Autónomo de México
43 c
44 c     Jorge Nocedal and Jose Luis Morales
45 c     Jorge Nocedal and Jose Luis Morales, Remark on "Algorithm 778:
46 c     L-BFGS-B: Fortran Subroutines for Large-Scale Bound Constrained
47 c     Optimization" (2011). To appear in ACM Transactions on
48 c     Mathematical Software,
49 c
50 c *****
51
52 program driver
53 use json_writer
54
55 c This driver shows how to replace the default stopping test
56 c by other termination criteria. It also illustrates how to
57 c print the values of several parameters during the course of
58 c the iteration. The sample problem used here is the same as in
59 c DRIVER1 (the extended Rosenbrock function with bounds on the
60 c variables).
61
62 integer          nmax, mmax
63 parameter(nmax=1024, mmax=17)
64 c     nmax is the dimension of the largest problem to be solved.
65 c     mmax is the maximum number of limited memory corrections.
66

```

```

67 c      Declare the variables needed by the code.
68 c      A description of all these variables is given at the end of
69 c      driver1.
70
71      character*60      task, csave
72      logical          lsave(4)
73      integer          n, m, iprint,
74      +                nbd(nmax), iwa(3*nmax), isave(44)
75      double precision f, factr, pgtol,
76      +                x(nmax), l(nmax), u(nmax), g(nmax), dsave(29),
77      +                wa(2*mmax*nmax+5*nmax+11*mmax*mmax+8*mmax)
78
79 c      Declare a few additional variables for the sample problem.
80
81      double precision t1, t2
82      integer          i
83
84 c      Test instrumentation: dump aggregate end-of-run state to JSON when
85 c      the environment variable LBFGSB_JSON_OUTPUT is set. No-op otherwise.
86 c      LBFGSB_NFG_LIMIT optionally overrides the nfg stopping threshold.
87      character*512     lbfgsb_json
88      character*64      env_nfg_limit
89      logical          json_active
90      integer          nfg_limit
91
92 c      We suppress the default output.
93
94      iprint = -1
95
96 c      We suppress both code-supplied stopping tests because the
97 c      user is providing his own stopping criteria.
98
99      factr=0.0d0
100     pgtol=0.0d0
101
102 c      We specify the dimension n of the sample problem and the number
103 c      m of limited memory corrections stored. (n and m should not
104 c      exceed the limits nmax and mmax respectively.)
105
106     n=25
107     m=5
108
109 c      We now specify nbd which defines the bounds on the variables:
110 c      l specifies the lower bounds,
111 c      u specifies the upper bounds.
112
113 c      First set bounds on the odd numbered variables.
114
115     do 10 i=1,n,2
116         nbd(i)=2
117         l(i)=1.0d0
118         u(i)=1.0d2
119 10    continue
120
121 c      Next set bounds on the even numbered variables.
122
123     do 12 i=2,n,2
124         nbd(i)=2
125         l(i)=-1.0d2
126         u(i)=1.0d2
127 12    continue
128
129 c      We now define the starting point.
130
131     do 14 i=1,n
132         x(i)=3.0d0
133 14    continue
134
135 c      We now write the heading of the output.
136
137     write (6,16)
138 16    format(/,5x, 'Solving sample problem.',
139      +      /,5x, ' (f = 0.0 at the optimal solution.)',/)
140
141 c      We start the iteration by initializing task.
142 c
143     task = 'START'
144
145 c      Test instrumentation: enabled when LBFGSB_JSON_OUTPUT is set.
146     call get_environment_variable('LBFGSB_JSON_OUTPUT', lbfgsb_json)
147     json_active = (len_trim(lbfgsb_json) .gt. 0)
148     nfg_limit = 99
149     call get_environment_variable('LBFGSB_NFG_LIMIT', env_nfg_limit)
150     if (len_trim(env_nfg_limit) .gt. 0) then
151         read(env_nfg_limit, *) nfg_limit
152     endif
153

```

```

154 c      ----- the beginning of the loop -----
155
156 111 continue
157
158 c      This is the call to the L-BFGS-B code.
159
160 call setulb(n,m,x,l,u,nbd,f,g,factr,pgtol,wa,iwa,task,iprint,
161 +         csave,lsave,isave,dsave)
162
163 if (task(1:2) .eq. 'FG') then
164 c      the minimization routine has returned to request the
165 c      function f and gradient g values at the current x.
166
167 c      Compute function value f for the sample problem.
168
169 f=.25d0*(x(1)-1.d0)**2
170 do 20 i=2,n
171     f=f+(x(i)-x(i-1)**2)**2
172 20 continue
173     f=4.d0*f
174
175 c      Compute gradient g for the sample problem.
176
177 t1=x(2)-x(1)**2
178 g(1)=2.d0*(x(1)-1.d0)-1.6d1*x(1)*t1
179 do 22 i=2,n-1
180     t2=t1
181     t1=x(i+1)-x(i)**2
182     g(i)=8.d0*t2-1.6d1*x(i)*t1
183 22 continue
184     g(n)=8.d0*t1
185
186 c      go back to the minimization routine.
187 goto 111
188 endif
189 c
190 if (task(1:5) .eq. 'NEW_X') then
191 c
192 c      the minimization routine has returned with a new iterate.
193 c      At this point have the opportunity of stopping the iteration
194 c      or observing the values of certain parameters
195 c
196 c      First are two examples of stopping tests.
197
198 c      Note: task(1:4) must be assigned the value 'STOP' to terminate
199 c      the iteration and ensure that the final results are
200 c      printed in the default format. The rest of the character
201 c      string TASK may be used to store other information.
202
203 c      1) Terminate if the total number of f and g evaluations
204 c      exceeds 99.
205
206 if (isave(34) .ge. nfg_limit)
207 +   task='STOP: TOTAL NO. of f AND g EVALUATIONS EXCEEDS LIMIT'
208
209 c      2) Terminate if |proj g|/(1+|f|) < 1.0d-10, where
210 c      "proj g" denoted the projected gradient
211
212 if (dsave(13) .le. 1.d-10*(1.0d0 + abs(f)))
213 +   task='STOP: THE PROJECTED GRADIENT IS SUFFICIENTLY SMALL'
214
215 c      We now wish to print the following information at each
216 c      iteration:
217 c
218 c      1) the current iteration number, isave(30),
219 c      2) the total number of f and g evaluations, isave(34),
220 c      3) the value of the objective function f,
221 c      4) the norm of the projected gradient, dsve(13)
222 c
223 c      See the comments at the end of driver1 for a description
224 c      of the variables isave and dsave.
225
226 write (6,' (2(a,i5,4x),a,1p,d12.5,4x,a,1p,d12.5)') 'Iterate'
227 +   ,isave(30),'nfg =',isave(34),'f =',f,'|proj g| =',dsave(13)
228
229 c      If the run is to be terminated, we print also the information
230 c      contained in task as well as the final value of x.
231
232 if (task(1:4) .eq. 'STOP') then
233     write (6,*) task
234     write (6,*) 'Final X='
235     write (6,' ((1x,1p, 6(1x,d11.4)))') (x(i),i = 1,n)
236 endif
237
238 c      go back to the minimization routine.
239 goto 111
240

```

```

241         endif
242
243 c         ----- the end of the loop -----
244
245 c         If task is neither FG nor NEW_X we terminate execution.
246
247         if (json_active) then
248             call json_write_aggregate(trim(lbfgsb_json), task, f,
249 +                                     dsave(13), n, x)
250         endif
251
252         stop
253
254     end
255
256 c===== The end of driver2 =====
257

```

4.4 driver2.f90

This driver shows how to replace the default stopping test by other termination criteria. It also illustrates how to print the values of several parameters during the course of the iteration. The sample problem used here is the same as in DRIVER1 (the extended Rosenbrock function with bounds on the variables). (Fortran-90 version)

```

1 !> \file driver2.f90
2
3 !
4 ! L-BFGS-B is released under the "New BSD License" (aka "Modified BSD License"
5 ! or "3-clause license")
6 ! Please read attached file License.txt
7 !
8 !
9 !
10 !             DRIVER 2   in Fortran 90
11 ! -----
12 !             CUSTOMIZED DRIVER FOR L-BFGS-B
13 ! -----
14 !
15 !     L-BFGS-B is a code for solving large nonlinear optimization
16 !     problems with simple bounds on the variables.
17 !
18 !     The code can also be used for unconstrained problems and is
19 !     as efficient for these problems as the earlier limited memory
20 !     code L-BFGS.
21 !
22 !     This driver illustrates how to control the termination of the
23 !     run and how to design customized output.
24 !
25 ! References:
26 !
27 !     [1] R. H. Byrd, P. Lu, J. Nocedal and C. Zhu, "A limited
28 !     memory algorithm for bound constrained optimization",
29 !     SIAM J. Scientific Computing 16 (1995), no. 5, pp. 1190--1208.
30 !
31 !     [2] C. Zhu, R.H. Byrd, P. Lu, J. Nocedal, "L-BFGS-B: FORTRAN
32 !     Subroutines for Large Scale Bound Constrained Optimization"
33 !     Tech. Report, NAM-11, EECS Department, Northwestern University,
34 !     1994.
35 !
36 !     (Postscript files of these papers are available via anonymous
37 !     ftp to eecs.nwu.edu in the directory pub/lbfgs/lbfgs_bcm.)
38 !
39 !             * * *
40 !
41 !     February 2011   (latest revision)
42 !     Optimization Center at Northwestern University
43 !     Instituto Tecnológico Autónomo de México
44 !
45 !     Jorge Nocedal and Jose Luis Morales
46 !
47 ! *****
48 ! program driver
49 ! use json_writer
50 !
51 ! This driver shows how to replace the default stopping test
52 ! by other termination criteria. It also illustrates how to
53 ! print the values of several parameters during the course of
54 ! the iteration. The sample problem used here is the same as in
55 ! DRIVER1 (the extended Rosenbrock function with bounds on the
56 ! variables).
57 !
58 ! implicit none

```

```

59
60 !      Declare variables and parameters needed by the code.
61 !
62 !      Note that we suppress the default output (iprint = -1)
63 !      We suppress both code-supplied stopping tests because the
64 !      user is providing his/her own stopping criteria.
65
66      integer, parameter      :: n = 25, m = 5, iprint = -1
67      integer, parameter      :: dp = kind(1.0d0)
68      real(dp), parameter     :: factr = 0.0d0, pgtol = 0.0d0
69
70      character(len=60)       :: task, csave
71      logical                  :: lsave(4)
72      integer                  :: isave(44)
73      real(dp)                 :: f
74      real(dp)                 :: dsave(29)
75      integer, allocatable     :: nbd(:), iwa(:)
76      real(dp), allocatable    :: x(:), l(:), u(:), g(:), wa(:)
77 !
78      real(dp)                 :: t1, t2
79      integer                  :: i
80
81 !      Test instrumentation: dump aggregate end-of-run state to JSON when
82 !      the environment variable LBFGSB_JSON_OUTPUT is set. No-op otherwise.
83 !      LBFGSB_NFG_LIMIT optionally overrides the nfg stopping threshold.
84      character(len=512)       :: lbfgsb_json
85      character(len=64)        :: env_nfg_limit
86      logical                  :: json_active
87      integer                  :: nfg_limit
88
89      allocate ( nbd(n), x(n), l(n), u(n), g(n) )
90      allocate ( iwa(3*n) )
91      allocate ( wa(2*m*n + 5*n + 11*m*m + 8*m) )
92 !
93 !      This driver shows how to replace the default stopping test
94 !      by other termination criteria. It also illustrates how to
95 !      print the values of several parameters during the course of
96 !      the iteration. The sample problem used here is the same as in
97 !      DRIVER1 (the extended Rosenbrock function with bounds on the
98 !      variables).
99 !      We now specify nbd which defines the bounds on the variables:
100 !          l    specifies the lower bounds,
101 !          u    specifies the upper bounds.
102
103 !      First set bounds on the odd numbered variables.
104
105      do 10 i=1, n,2
106          nbd(i)=2
107          l(i)=1.0d0
108          u(i)=1.0d2
109 10  continue
110
111 !      Next set bounds on the even numbered variables.
112
113      do 12 i=2, n,2
114          nbd(i)=2
115          l(i)=-1.0d2
116          u(i)=1.0d2
117 12  continue
118
119 !      We now define the starting point.
120
121      do 14 i=1, n
122          x(i)=3.0d0
123 14  continue
124
125 !      We now write the heading of the output.
126
127      write (6,16)
128 16  format(/,5x, 'Solving sample problem.', &
129          /,5x, ' (f = 0.0 at the optimal solution.)',/)
130
131
132 !      We start the iteration by initializing task.
133 !
134      task = 'START'
135
136 !      Test instrumentation: enabled when LBFGSB_JSON_OUTPUT is set.
137      call get_environment_variable('LBFGSB_JSON_OUTPUT', lbfgsb_json)
138      json_active = (len_trim(lbfgsb_json) > 0)
139      nfg_limit = 99
140      call get_environment_variable('LBFGSB_NFG_LIMIT', env_nfg_limit)
141      if (len_trim(env_nfg_limit) > 0) read(env_nfg_limit, *) nfg_limit
142
143 !      ----- the beginning of the loop -----
144
145      do while( task(1:2).eq.'FG'.or.task.eq.'NEW_X'.or. &

```

```

146         task.eq.'START')
147
148 !      This is the call to the L-BFGS-B code.
149
150         call setulb(n,m,x,l,u,nbd,f,g,factr,pgtol,wa,iwa,task,iprint, &
151                  csave,lsave,isave,dsave)
152
153         if (task(1:2) .eq. 'FG') then
154
155 !      the minimization routine has returned to request the
156 !      function f and gradient g values at the current x.
157
158 !      Compute function value f for the sample problem.
159
160         f = .25d0*(x(1) - 1.d0)**2
161         do 20 i=2,n
162             f = f + (x(i) - x(i-1)**2)**2
163         20 continue
164         f = 4.d0*f
165
166 !      Compute gradient g for the sample problem.
167
168         t1 = x(2) - x(1)**2
169         g(1) = 2.d0*(x(1) - 1.d0) - 1.6d1*x(1)*t1
170         do 22 i= 2,n-1
171             t2 = t1
172             t1 = x(i+1) - x(i)**2
173             g(i) = 8.d0*t2 - 1.6d1*x(i)*t1
174         22 continue
175         g(n)=8.d0*t1
176 !
177     else
178 !
179         if (task(1:5) .eq. 'NEW_X') then
180
181 !      the minimization routine has returned with a new iterate.
182 !      At this point have the opportunity of stopping the iteration
183 !      or observing the values of certain parameters
184 !
185 !      First are two examples of stopping tests.
186
187 !      Note: task(1:4) must be assigned the value 'STOP' to terminate
188 !      the iteration and ensure that the final results are
189 !      printed in the default format. The rest of the character
190 !      string TASK may be used to store other information.
191
192 !      1) Terminate if the total number of f and g evaluations
193 !      exceeds 99.
194
195         if (isave(34) .ge. nfg_limit) &
196             task='STOP: TOTAL NO. of f AND g EVALUATIONS EXCEEDS LIMIT'
197
198 !      2) Terminate if  $|proj\ g|/(1+|f|) < 1.0d-10$ , where
199 !      "proj g" denoted the projected gradient
200
201         if (dsave(13) .le. 1.d-10*(1.0d0 + abs(f))) &
202             task='STOP: THE PROJECTED GRADIENT IS SUFFICIENTLY SMALL'
203
204 !      We now wish to print the following information at each
205 !      iteration:
206 !
207 !      1) the current iteration number, isave(30),
208 !      2) the total number of f and g evaluations, isave(34),
209 !      3) the value of the objective function f,
210 !      4) the norm of the projected gradient, dsve(13)
211 !
212 !      See the comments at the end of driver1 for a description
213 !      of the variables isave and dsave.
214
215         write (6,' (2(a,i5,4x),a,1p,d12.5,4x,a,1p,d12.5)') 'Iterate' &
216             , isave(30),nfg =',isave(34),'f =',f,'|proj g| =',dsave(13)
217
218 !      If the run is to be terminated, we print also the information
219 !      contained in task as well as the final value of x.
220
221         if (task(1:4) .eq. 'STOP') then
222             write (6,*) task
223             write (6,*) 'Final X='
224             write (6,' ((1x,1p, 6(1x,d11.4)))') (x(i),i = 1,n)
225         end if
226     end if
227 end if
228 end if
229 end do
230
231 !      ----- the end of the loop -----
232

```

```

233 !      If task is neither FG nor NEW_X we terminate execution.
234
235      if (json_active) &
236          call json_write_aggregate(trim(lbfgsb_json), task, f, dsave(13), n, x)
237
238      end program driver
239
240 !===== The end of driver2 =====
241

```

4.5 driver3.f

This time-controlled driver shows that it is possible to terminate a run by elapsed CPU time, and yet be able to print all desired information. This driver also illustrates the use of two stopping criteria that may be used in conjunction with a limit on execution time. The sample problem used here is the same as in driver1 and driver2 (the extended Rosenbrock function with bounds on the variables). (Fortran-77 version)

```

1 c> \file driver3.f
2
3 c
4 c  L-BFGS-B is released under the "New BSD License" (aka "Modified BSD License"
5 c  or "3-clause license")
6 c  Please read attached file License.txt
7 c
8 c                                DRIVER 3 in Fortran 77
9 c  -----
10 c      TIME-CONTROLLED DRIVER FOR L-BFGS-B (version 3.0)
11 c  -----
12 c
13 c      L-BFGS-B is a code for solving large nonlinear optimization
14 c      problems with simple bounds on the variables.
15 c
16 c      The code can also be used for unconstrained problems and is
17 c      as efficient for these problems as the earlier limited memory
18 c      code L-BFGS.
19 c
20 c      This driver shows how to terminate a run after some prescribed
21 c      CPU time has elapsed, and how to print the desired information
22 c      before exiting.
23 c
24 c  References:
25 c
26 c      [1] R. H. Byrd, P. Lu, J. Nocedal and C. Zhu, "A limited
27 c      memory algorithm for bound constrained optimization",
28 c      SIAM J. Scientific Computing 16 (1995), no. 5, pp. 1190--1208.
29 c
30 c      [2] C. Zhu, R.H. Byrd, P. Lu, J. Nocedal, "L-BFGS-B: FORTRAN
31 c      Subroutines for Large Scale Bound Constrained Optimization"
32 c      Tech. Report, NAM-11, EECS Department, Northwestern University,
33 c      1994.
34 c
35 c
36 c      (Postscript files of these papers are available via anonymous
37 c      ftp to eeecs.nwu.edu in the directory pub/lbfgs/lbfgs_bcm.)
38 c
39 c      * * *
40 c
41 c      February 2011    (latest revision)
42 c      Optimization Center at Northwestern University
43 c      Instituto Tecnológico Autónomo de México
44 c
45 c      Jorge Nocedal and Jose Luis Morales, Remark on "Algorithm 778:
46 c      L-BFGS-B: Fortran Subroutines for Large-Scale Bound Constrained
47 c      Optimization" (2011). To appear in ACM Transactions on
48 c      Mathematical Software,
49 c
50 c
51 c  *****
52
53  program driver
54  use json_writer
55
56  This time-controlled driver shows that it is possible to terminate
57  a run by elapsed CPU time, and yet be able to print all desired
58  information. This driver also illustrates the use of two
59  stopping criteria that may be used in conjunction with a limit
60  on execution time. The sample problem used here is the same as in
61  driver1 and driver2 (the extended Rosenbrock function with bounds
62  on the variables).
63
64  integer          nmax, mmax
65  parameter(nmax=1024,mmax=17)

```

```

66 c      nmax is the dimension of the largest problem to be solved.
67 c      mmax is the maximum number of limited memory corrections.
68
69 c      Declare the variables needed by the code.
70 c      A description of all these variables is given at the end of
71 c      driver1.
72
73 character*60      task, csave
74 logical          lsave(4)
75 integer          n, m, iprint,
76 +               nbd(nmax), iwa(3*nmax), isave(44)
77 double precision f, factr, pgtol,
78 +               x(nmax), l(nmax), u(nmax), g(nmax), dsave(29),
79 +               wa(2*mmax*nmax+5*nmax+11*mmax*mmax+8*mmax)
80
81 c      Declare a few additional variables for the sample problem
82 c      and for keeping track of time.
83
84 double precision t1, t2, time1, time2, tlimit
85 integer          i, j
86
87 c      Test instrumentation: dump per-iteration state to JSON when the
88 c      environment variable LBFGSB_JSON_OUTPUT is set. No-op otherwise.
89 c      LBFGSB_TLIMIT optionally overrides tlimit for reproducibility.
90 character*512      lbfgsb_json
91 character*64       env_tlimit
92 logical           json_active
93
94 c      We specify a limite on the CPU time (in seconds).
95
96 tlimit = 0.2
97 call get_environment_variable('LBFGSB_TLIMIT', env_tlimit)
98 if (len_trim(env_tlimit) .gt. 0) read(env_tlimit, *) tlimit
99
100 c      We suppress the default output. (The user could also elect to
101 c      use the default output by choosing iprint >= 0.)
102
103 iprint = -1
104
105 c      We suppress the code-supplied stopping tests because we will
106 c      provide our own termination conditions
107
108 factr=0.0d0
109 pgtol=0.0d0
110
111 c      We specify the dimension n of the sample problem and the number
112 c      m of limited memory corrections stored. (n and m should not
113 c      exceed the limits nmax and mmax respectively.)
114
115 n=1000
116 m=10
117
118 c      We now specify nbd which defines the bounds on the variables:
119 c      l specifies the lower bounds,
120 c      u specifies the upper bounds.
121
122 c      First set bounds on the odd-numbered variables.
123
124 do 10 i=1,n,2
125     nbd(i)=2
126     l(i)=1.0d0
127     u(i)=1.0d2
128 10 continue
129
130 c      Next set bounds on the even-numbered variables.
131
132 do 12 i=2,n,2
133     nbd(i)=2
134     l(i)=-1.0d2
135     u(i)=1.0d2
136 12 continue
137
138 c      We now define the starting point.
139
140 do 14 i=1,n
141     x(i)=3.0d0
142 14 continue
143
144 c      We now write the heading of the output.
145
146 write (6,16)
147 16 format(/,5x, 'Solving sample problem.',
148 +       /,5x, ' (f = 0.0 at the optimal solution.)',/)
149
150 c      We start the iteration by initializing task.
151 c
152 task = 'START'

```

```

153
154 c      Test instrumentation: enabled when LBFGSB_JSON_OUTPUT is set.
155 call get_environment_variable('LBFGSB_JSON_OUTPUT', lbfgsb_json)
156 json_active = (len_trim(lbfgsb_json) .gt. 0)
157
158 c      ----- the beginning of the loop -----
159
160 c      We begin counting the CPU time.
161
162 call timer(time1)
163
164 111 continue
165
166 c      This is the call to the L-BFGS-B code.
167
168 call setulb(n,m,x,l,u,nbd,f,g,factr,pgtol,wa,iwa,task,iprint,
169 +          csave,lsave,isave,dsave)
170
171 if (task(1:2) .eq. 'FG') then
172 c      the minimization routine has returned to request the
173 c      function f and gradient g values at the current x.
174 c      Before evaluating f and g we check the CPU time spent.
175
176 call timer(time2)
177 if (time2-time1 .gt. tlimit) then
178     task='STOP: CPU EXCEEDING THE TIME LIMIT.'
179
180 c      Note: Assigning task(1:4)='STOP' will terminate the run;
181 c      setting task(7:9)='CPU' will restore the information at
182 c      the latest iterate generated by the code so that it can
183 c      be correctly printed by the driver.
184
185 c      In this driver we have chosen to disable the
186 c      printing options of the code (we set iprint=-1);
187 c      instead we are using customized output: we print the
188 c      latest value of x, the corresponding function value f and
189 c      the norm of the projected gradient |proj g|.
190
191 c      We print out the information contained in task.
192
193 write (6,*) task
194
195 c      We print the latest iterate contained in wa(j+1:j+n), where
196 c
197     j = 3*n+2*m*n+11*m**2
198     write (6,*) 'Latest iterate X ='
199     write (6,'((1x,1p, 6(1x,d11.4)))') (wa(i),i = j+1,j+n)
200
201 c      We print the function value f and the norm of the projected
202 c      gradient |proj g| at the last iterate; they are stored in
203 c      dsave(2) and dsave(13) respectively.
204
205     write (6,'(a,1p,d12.5,4x,a,1p,d12.5)')
206 +     'At latest iterate   f =',dsave(2),'|proj g| =',dsave(13)
207
208 else
209
210 c      The time limit has not been reached and we compute
211 c      the function value f for the sample problem.
212
213     f=.25d0*(x(1)-1.d0)**2
214     do 20 i=2,n
215         f=f+(x(i)-x(i-1)**2)**2
216 20    continue
217     f=4.d0*f
218
219 c      Compute gradient g for the sample problem.
220
221     t1=x(2)-x(1)**2
222     g(1)=2.d0*(x(1)-1.d0)-1.6d1*x(1)*t1
223     do 22 i=2,n-1
224         t2=t1
225         t1=x(i+1)-x(i)**2
226         g(i)=8.d0*t2-1.6d1*x(i)*t1
227 22    continue
228     g(n)=8.d0*t1
229
230 endif
231
232 c      go back to the minimization routine.
233 goto 111
234 endif
235
236 c      if (task(1:5) .eq. 'NEW_X') then
237 c      the minimization routine has returned with a new iterate.
238 c      The time limit has not been reached, and we test whether
239 c      the following two stopping tests are satisfied:

```

```

240
241 c      1) Terminate if the total number of f and g evaluations
242 c      exceeds 900.
243
244      if (isave(34) .ge. 900)
245 +      task='STOP: TOTAL NO. of f AND g EVALUATIONS EXCEEDS LIMIT'
246
247 c      2) Terminate if |proj g|/(1+|f|) < 1.0d-10.
248
249      if (dsave(13) .le. 1.d-10*(1.0d0 + abs(f)))
250 +      task='STOP: THE PROJECTED GRADIENT IS SUFFICIENTLY SMALL'
251
252 c      We wish to print the following information at each iteration:
253 c      1) the current iteration number, isave(30),
254 c      2) the total number of f and g evaluations, isave(34),
255 c      3) the value of the objective function f,
256 c      4) the norm of the projected gradient, dsve(13)
257 c
258 c      See the comments at the end of driver1 for a description
259 c      of the variables isave and dsave.
260
261
262      write (6,'(2(a,i5,4x),a,1p,d12.5,4x,a,1p,d12.5)') 'Iterate'
263 +      ,isave(30),'nfg =',isave(34),'f =',f,'|proj g| =',dsave(13)
264
265 c      If the run is to be terminated, we print also the information
266 c      contained in task as well as the final value of x.
267
268
269      if (task(1:4) .eq. 'STOP') then
270          write (6,*) task
271          write (6,*) 'Final X='
272          write (6,'((1x,1p, 6(1x,d11.4)))') (x(i),i = 1,n)
273      endif
274
275 c      go back to the minimization routine.
276 c      goto 111
277
278      endif
279
280 c      ----- the end of the loop -----
281
282 c      If task is neither FG nor NEW_X we terminate execution.
283
284      if (json_active) then
285          call json_write_aggregate(trim(lbfgsb_json), task, f,
286 +      dsave(13), n, x)
287      endif
288
289      stop
290
291      end
292
293 c===== The end of driver3 =====
294

```

4.6 driver3.f90

This time-controlled driver shows that it is possible to terminate a run by elapsed CPU time, and yet be able to print all desired information. This driver also illustrates the use of two stopping criteria that may be used in conjunction with a limit on execution time. The sample problem used here is the same as in driver1 and driver2 (the extended Rosenbrock function with bounds on the variables). (Fortran-90 version)

```

1 !> \file driver3.f90
2
3 !
4 ! L-BFGS-B is released under the "New BSD License" (aka "Modified BSD License"
5 ! or "3-clause license")
6 ! Please read attached file License.txt
7 !
8 !          DRIVER 3   in Fortran 90
9 ! -----
10 !      TIME-CONTROLLED DRIVER FOR L-BFGS-B
11 ! -----
12 !
13 !      L-BFGS-B is a code for solving large nonlinear optimization
14 !      problems with simple bounds on the variables.
15 !
16 !      The code can also be used for unconstrained problems and is
17 !      as efficient for these problems as the earlier limited memory
18 !      code L-BFGS.
19 !

```

```

20 !      This driver shows how to terminate a run after some prescribed
21 !      CPU time has elapsed, and how to print the desired information
22 !      before exiting.
23 !
24 !      References:
25 !
26 !      [1] R. H. Byrd, P. Lu, J. Nocedal and C. Zhu, "A limited
27 !      memory algorithm for bound constrained optimization",
28 !      SIAM J. Scientific Computing 16 (1995), no. 5, pp. 1190--1208.
29 !
30 !      [2] C. Zhu, R.H. Byrd, P. Lu, J. Nocedal, "L-BFGS-B: FORTRAN
31 !      Subroutines for Large Scale Bound Constrained Optimization"
32 !      Tech. Report, NAM-11, EECS Department, Northwestern University,
33 !      1994.
34 !
35 !
36 !      (Postscript files of these papers are available via anonymous
37 !      ftp to eecs.nwu.edu in the directory pub/lbfgs/lbfgs_bcm.)
38 !
39 !      * * *
40 !
41 !      February 2011 (latest revision)
42 !      Optimization Center at Northwestern University
43 !      Instituto Tecnológico Autónomo de México
44 !
45 !      Jorge Nocedal and Jose Luis Morales
46 !
47 !      *****
48
49 program driver
50 use json_writer
51
52 !      This time-controlled driver shows that it is possible to terminate
53 !      a run by elapsed CPU time, and yet be able to print all desired
54 !      information. This driver also illustrates the use of two
55 !      stopping criteria that may be used in conjunction with a limit
56 !      on execution time. The sample problem used here is the same as in
57 !      driver1 and driver2 (the extended Rosenbrock function with bounds
58 !      on the variables).
59
60 implicit none
61
62 !      We specify a limit on the CPU time (tlimit = 10 seconds)
63 !
64 !      We suppress the default output (iprint = -1). The user could
65 !      also elect to use the default output by choosing iprint >= 0.)
66 !      We suppress the code-supplied stopping tests because we will
67 !      provide our own termination conditions
68 !      We specify the dimension n of the sample problem and the number
69 !      m of limited memory corrections stored.
70
71 integer, parameter :: n = 1000, m = 10, iprint = -1
72 integer, parameter :: dp = kind(1.0d0)
73 real(dp), parameter :: factr = 0.0d0, pgtol = 0.0d0
74 real(dp) :: tlimit
75 !
76 character(len=60) :: task, csave
77 logical :: lsave(4)
78 integer :: isave(44)
79 real(dp) :: f
80 real(dp) :: dsave(29)
81 integer, allocatable :: nbd(:), iwa(:)
82 real(dp), allocatable :: x(:), l(:), u(:), g(:), wa(:)
83 !
84 real(dp) :: t1, t2, time1, time2
85 integer :: i, j
86
87 !      Test instrumentation: dump per-iteration state to JSON when the
88 !      environment variable LBFGSB_JSON_OUTPUT is set. No-op otherwise.
89 !      LBFGSB_TLIMIT optionally overrides tlimit for reproducibility.
90 character(len=512) :: lbfgsb_json
91 character(len=64) :: env_tlimit
92 logical :: json_active
93
94 allocate ( nbd(n), x(n), l(n), u(n), g(n) )
95 allocate ( iwa(3*n) )
96 allocate ( wa(2*m*n + 5*n + 11*m*m + 8*m) )
97
98 !      This time-controlled driver shows that it is possible to terminate
99 !      a run by elapsed CPU time, and yet be able to print all desired
100 !      information. This driver also illustrates the use of two
101 !      stopping criteria that may be used in conjunction with a limit
102 !      on execution time. The sample problem used here is the same as in
103 !      driver1 and driver2 (the extended Rosenbrock function with bounds
104 !      on the variables).
105
106 !      We now specify nbd which defines the bounds on the variables:

```

```

107 !           l   specifies the lower bounds,
108 !           u   specifies the upper bounds.
109
110 !   First set bounds on the odd-numbered variables.
111
112       do 10 i=1, n,2
113           nbd(i)=2
114           l(i)=1.0d0
115           u(i)=1.0d2
116 10    continue
117
118 !   Next set bounds on the even-numbered variables.
119
120       do 12 i=2, n,2
121           nbd(i)=2
122           l(i)=-1.0d2
123           u(i)=1.0d2
124 12    continue
125
126 !   We now define the starting point.
127
128       do 14 i=1, n
129           x(i)=3.0d0
130 14    continue
131
132 !   We now write the heading of the output.
133
134       write (6,16)
135 16    format(/,5x, 'Solving sample problem.', &
136           /,5x, ' (f = 0.0 at the optimal solution.)',/)
137
138 !   We start the iteration by initializing task.
139
140       task = 'START'
141
142 !   Initialise tlimit (with optional override from LBFGSB_TLIMIT) and
143 !   enable JSON output if LBFGSB_JSON_OUTPUT is set.
144       tlimit = 10.0d0
145       call get_environment_variable('LBFGSB_TLIMIT', env_tlimit)
146       if (len_trim(env_tlimit) > 0) read(env_tlimit, *) tlimit
147       call get_environment_variable('LBFGSB_JSON_OUTPUT', lbfgsb_json)
148       json_active = (len_trim(lbfgsb_json) > 0)
149
150 !   ----- the beginning of the loop -----
151
152 !   We begin counting the CPU time.
153
154       call timer(time1)
155
156       do while( task(1:2).eq.'FG'.or.task.eq.'NEW_X'.or. &
157              task.eq.'START')
158
159 !   This is the call to the L-BFGS-B code.
160
161           call setulb(n,m,x,l,u,nbd,f,g,factr,pgtol,wa,iwa, &
162                    task,iprint, csave,lsave,isave,dsave)
163
164           if (task(1:2) .eq. 'FG') then
165
166 !   the minimization routine has returned to request the
167 !   function f and gradient g values at the current x.
168 !   Before evaluating f and g we check the CPU time spent.
169
170           call timer(time2)
171           if (time2-time1 .gt. tlimit) then
172               task='STOP: CPU EXCEEDING THE TIME LIMIT.'
173
174 !   Note: Assigning task(1:4)='STOP' will terminate the run;
175 !   setting task(7:9)='CPU' will restore the information at
176 !   the latest iterate generated by the code so that it can
177 !   be correctly printed by the driver.
178
179 !   In this driver we have chosen to disable the
180 !   printing options of the code (we set iprint=-1);
181 !   instead we are using customized output: we print the
182 !   latest value of x, the corresponding function value f and
183 !   the norm of the projected gradient |proj g|.
184
185 !   We print out the information contained in task.
186
187           write (6,*) task
188
189 !   We print the latest iterate contained in wa(j+1:j+n), where
190
191           j = 3*n+2*m*n+11*m**2
192           write (6,*) 'Latest iterate X ='
193           write (6,'((1x,1p, 6(1x,d11.4)))') (wa(i),i = j+1,j+n)

```

```

194
195 !      We print the function value f and the norm of the projected
196 !      gradient |proj g| at the last iterate; they are stored in
197 !      dsave(2) and dsave(13) respectively.
198
199      write (6,' (a,lp,d12.5,4x,a,lp,d12.5)') &
200      'At latest iterate   f =',dsave(2),' |proj g| =',dsave(13)
201      else
202
203 !      The time limit has not been reached and we compute
204 !      the function value f for the sample problem.
205
206      f=.25d0*(x(1)-1.d0)**2
207      do 20 i=2, n
208          f=f+(x(i)-x(i-1)**2)**2
209 20      continue
210      f=4.d0*f
211
212 !      Compute gradient g for the sample problem.
213
214      t1 = x(2) - x(1)**2
215      g(1) = 2.d0*(x(1)-1.d0)-1.6d1*x(1)*t1
216      do 22 i=2,n-1
217          t2=t1
218          t1=x(i+1)-x(i)**2
219          g(i)=8.d0*t2-1.6d1*x(i)*t1
220 22      continue
221      g(n)=8.d0*t1
222      endif
223
224 !      go back to the minimization routine.
225      else
226
227          if (task(1:5) .eq. 'NEW_X') then
228
229 !      the minimization routine has returned with a new iterate.
230 !      The time limit has not been reached, and we test whether
231 !      the following two stopping tests are satisfied:
232
233 !      1) Terminate if the total number of f and g evaluations
234 !      exceeds 900.
235
236          if (isave(34) .ge. 900) &
237          task='STOP: TOTAL NO. of f AND g EVALUATIONS EXCEEDS LIMIT'
238
239 !      2) Terminate if |proj g|/(1+|f|) < 1.0d-10.
240
241          if (dsave(13) .le. 1.d-10*(1.0d0 + abs(f))) &
242          task='STOP: THE PROJECTED GRADIENT IS SUFFICIENTLY SMALL'
243
244 !      We wish to print the following information at each iteration:
245 !      1) the current iteration number, isave(30),
246 !      2) the total number of f and g evaluations, isave(34),
247 !      3) the value of the objective function f,
248 !      4) the norm of the projected gradient, dsve(13)
249 !
250 !      See the comments at the end of driver1 for a description
251 !      of the variables isave and dsave.
252
253      write (6,' (2(a,i5,4x),a,lp,d12.5,4x,a,lp,d12.5)') 'Iterate' &
254      ,isave(30),'nfg =',isave(34),'f =',f,' |proj g| =',dsave(13)
255
256 !      If the run is to be terminated, we print also the information
257 !      contained in task as well as the final value of x.
258
259      if (task(1:4) .eq. 'STOP') then
260          write (6,*) task
261          write (6,*) 'Final X='
262          write (6,' ((1x,lp, 6(1x,d11.4)))') (x(i),i = 1,n)
263      endif
264
265      endif
266      end if
267      end do
268
269 !      If task is neither FG nor NEW_X we terminate execution.
270
271      if (json_active) &
272      call json_write_aggregate(trim(lbfgsb_json), task, f, dsave(13), n, x)
273
274      end program driver
275
276 !====== The end of driver3 =====
277

```

Index

active
 active.f, 4
active.f
 active, 4

bmv
 bmv.f, 5
bmv.f
 bmv, 5

cauchy
 cauchy.f, 6
cauchy.f
 cauchy, 6
cmprib
 cmprib.f, 10
cmprib.f
 cmprib, 10

dcsrch
 dcsrch.f, 12
dcsrch.f
 dcsrch, 12
dcstep
 dcstep.f, 14
dcstep.f
 dcstep, 14

errclb
 errclb.f, 16
errclb.f
 errclb, 16

formk
 formk.f, 17
formk.f
 formk, 17
formt
 formt.f, 19
formt.f
 formt, 19
freev
 freev.f, 20
freev.f
 freev, 20

hpsolb
 hpsolb.f, 21
hpsolb.f
 hpsolb, 21

lnsrlb
 lnsrlb.f, 22
lnsrlb.f
 lnsrlb, 22

mainlb
 mainlb.f, 24
mainlb.f
 mainlb, 24
matupd
 matupd.f, 29
matupd.f
 matupd, 29

prn1lb
 prn1lb.f, 31
prn1lb.f
 prn1lb, 31
prn2lb
 prn2lb.f, 32
prn2lb.f
 prn2lb, 32
prn3lb
 prn3lb.f, 34
prn3lb.f
 prn3lb, 34
projgr
 projgr.f, 35
projgr.f
 projgr, 35

setulb
 setulb.f, 36
setulb.f
 setulb, 36
src/active.f, 4
src/bmv.f, 5
src/cauchy.f, 6
src/cmprib.f, 10
src/dcsrch.f, 11
src/dcstep.f, 14
src/errclb.f, 15
src/formk.f, 16
src/formt.f, 18
src/freev.f, 19
src/hpsolb.f, 21
src/lnsrlb.f, 22
src/mainlb.f, 24
src/matupd.f, 29
src/prn1lb.f, 30
src/prn2lb.f, 32
src/prn3lb.f, 33
src/projgr.f, 35
src/setulb.f, 36
src/subsm.f, 40
src/timer.f, 43
subsm
 subsm.f, 40
subsm.f
 subsm, 40

timer

timer.f, [43](#)
timer.f
 timer, [43](#)